



西安电子科技大学
XIDIAN UNIVERSITY



人工智能导论

状态空间表示及其搜索技术

主讲人：刘若辰

西安电子科技大学 人工智能学院



引言

- 人工智能的多个研究领域从求解现实问题的过程来看，都可抽象为一个“**问题求解**”过程，问题求解过程实际上就是一个搜索过程。
- 在搜索过程开始之前，必须先用某种方法或某几种方法的混和来表示问题。
- 问题求解技术主要涉及两个方面：
 - 问题的表示**
 - 求解的方法**
- **状态空间表示法**：用来表示问题及其搜索过程的一种方法。



迷宫问题

- 走迷宫是人们熟悉的一种游戏, 图2.1是一个9个格子迷宫。
- S_0 是迷宫入口, S_g 是出口
- 迷宫问题: 找出从入口到出口的一条路径
- 约束: 每次只能走一步

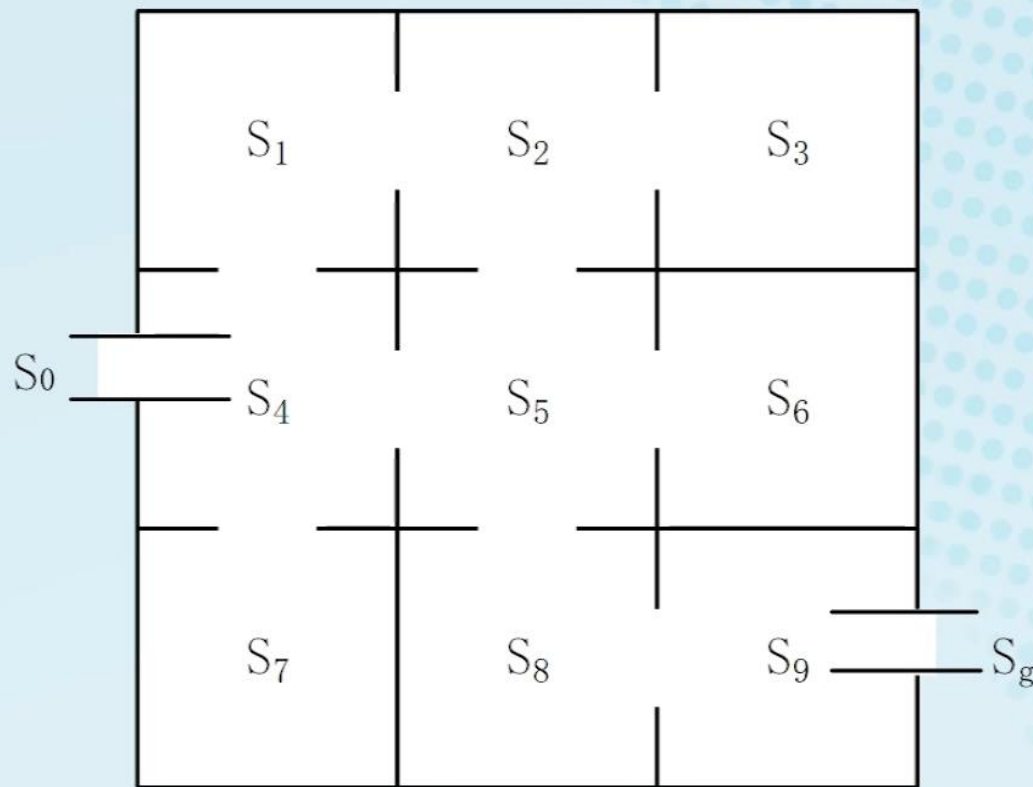


图2.1: 9格子迷宫

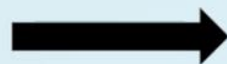


八数码难题

- 在 3×3 的棋盘，摆有八个棋子，每个棋子上标有1至8的某一数字。棋盘上还有一个空格，与空格相邻的棋子可以移到空格中。

2	8	3
1		4
7	6	5

初始状态



1	2	3
8		4
7	6	5

目标状态

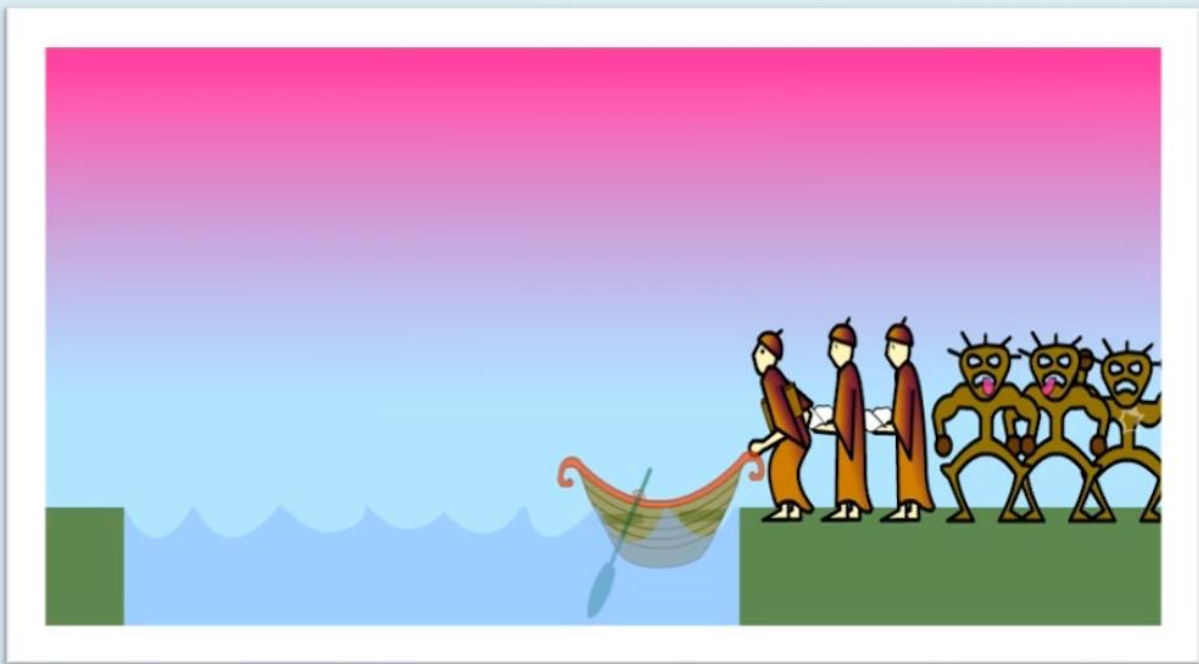
如何将棋盘从某一初始状态变成最后的目标状态？



传教士和野人问题

有三个传教士M和三个野人C过河，只有一条能装下两个人的船，在河的一方或者船上，如果野人的人数大于传教士的人数，那么传教士就会有危险。

能不能提出一种安全的渡河方法呢？





问题

- ◆ 调度
- ◆ 导航
- ◆ 分配
- ◆ 路径规划
- ◆ 游戏

-
-
-

问题有了，可怎么让计算机知道这些问题呢





问题的状态空间表示

- 状态空间方法：基于**解答空间的问题表示和求解方法**，它是以**状态和算符**为基础来表示和求解问题的。
- 主要包括：
 - **状态**
 - **算符**
 - **状态空间**
- 典型的例子：下棋、迷宫、各种游戏等。



问题的状态空间表示

- **状态** (State) : 表示问题求解过程中每一步问题状况的数据结构, 一般用一组数据表示:

$$S_k = \{S_{k_0}, S_{k_1}, \dots\}$$

式中每个元素为集合的分量, 称为状态变量

- 每一个分量给予确定的值时, 得到一个具体的状态。
- 任何一种类型的数据结构都可以用来描述状态, 只要它有利于问题求解。
- 在程序中用字符、数字、记录、数组、结构、对象等表示状态



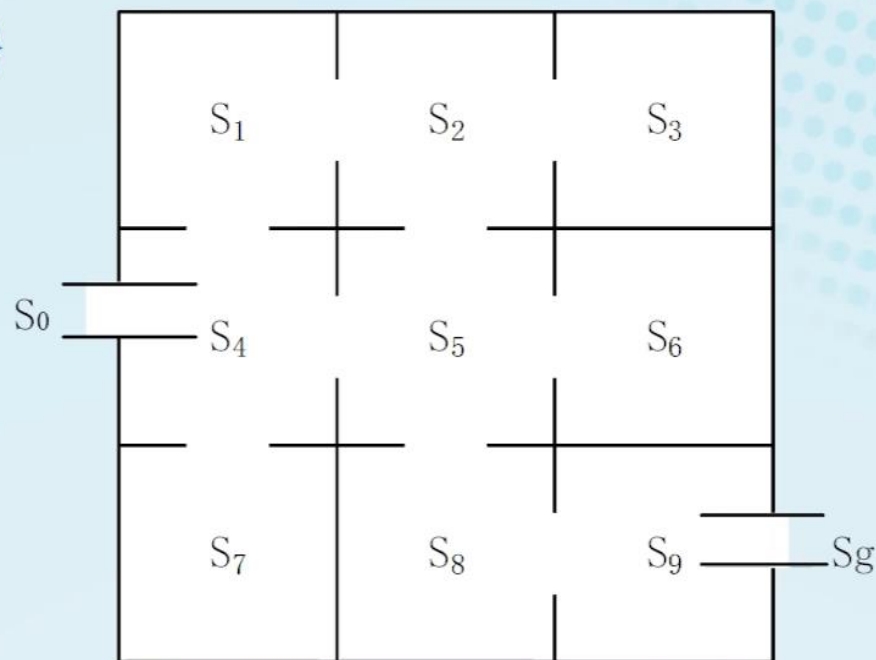
迷宫问题的状态

➤ 以每个格子作为一个状态，并用其标识符作为其表示。

- 该迷宫的状态可表示为： $S_k = (S_i)$
- S_i 可以取值为 $\{S_0, S_1, S_2, S_3, \dots, S_g\}$

➤ 初始状态： S_0

➤ 目标状态： S_g





八数码问题的状态

➤ 将棋局

表2.1

X_1	X_2	X_3
X_8	X_0	X_4
X_7	X_6	X_5

用向量 $S = (X_0, X_1, X_2, X_3, X_4, X_5, X_6, X_7, X_8)$ 表示状态

X_i 为变量， X_i 的值就是方格 X_i 内的数字，取值为 $\{0, 1, 2, \dots, 8\}$ ，
其中 0 表示空格。

于是，向量 S 就是该问题的状态表达式。

2	8	3
1		4
7	6	5

初始状态

1	2	3
8		4
7	6	5

目标状态

初始状态: $S_0 = (028345671)$

目标状态: $S_g = (012345678)$



问题的状态空间表示

- **算符 (Operator)**：当对一个问题状态使用某个可用操作时，它将引起该状态中**某些分量值的变化**，从而使问题从一个具体状态变为另一个具体状态。
- 算符可理解为状态集合上的一个函数，它描述了状态之间的关系。
 - 算符可以是某种操作、规则、行为、变换、函数、算子、过程等。
 - 算符也称为操作，问题的状态也只能经定义在其上的这种操作而改变。





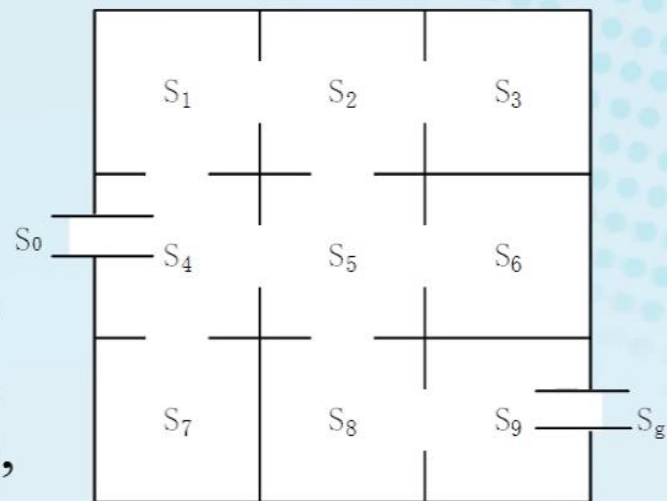
迷宫问题的算符

➤ 以每个格子作为一个状态,并用其标识符作为其表示。那么,两个标识符组成的序对就是一个算符。于是:

- 初始状态: S_0
- 该迷宫的算符 F :

$$F = \{(S_0, S_4), (S_4, S_0), (S_4, S_1), (S_1, S_4), (S_1, S_2), (S_2, S_1), (S_2, S_3), (S_3, S_2), (S_2, S_5), (S_5, S_2), (S_4, S_7), (S_7, S_4), (S_4, S_5), (S_5, S_4), (S_5, S_6), (S_6, S_5), (S_5, S_8), (S_8, S_5), (S_8, S_9), (S_9, S_8), (S_9, S_g)\}$$

- 目标状态: S_g





八数码难题的算符

对于八数码难题：

1		3
7	2	4
6	8	5

初始棋局



1	2	3
8		4
7	6	5

目标棋局

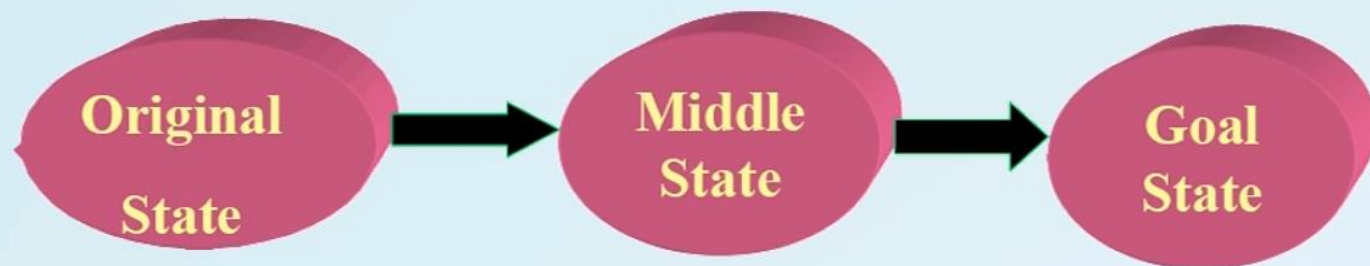
➤ 制定**操作算符集**：

- 直观方法——为每个棋牌制定一套可能的走步：**左、上、右、下**四种移动。这样就需**32个操作算子**。
- 简易方法——仅为空格制定这**4种走步**，因为只有紧靠空格的棋牌才能移动。空格移动的唯一约束是不能移出棋盘。



问题的状态空间表示

- **问题的状态空间 (State space)**：用来描述一个问题的全部状态以及这些状态之间的相互关系。
- 状态空间常用一个三元组表示：(S, F, G)
 - S：为问题的所有初始状态的集合；
 - F：算符的集合
 - G：为目标状态的集合

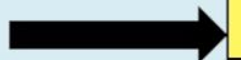




状态空间图

1		3
7	2	4
6	8	5

初始棋局



1	2	3
8		4
7	6	5

目标棋局

- 从初始棋局开始，试探由每一合法走步得到的各种新棋局，然后再走一步而得到的下一组棋局。这样继续下去，直至达到目标棋局为止。
- 把初始状态可达到的各状态所组成的空间设想为一幅由各种状态对应的节点组成的图。**这种图称为状态空间图。**
- 图中每个节点是它所代表的棋局。首先把适用的算符用于初始状态，以产生新的状态；算符可以看成状态空间图的边。

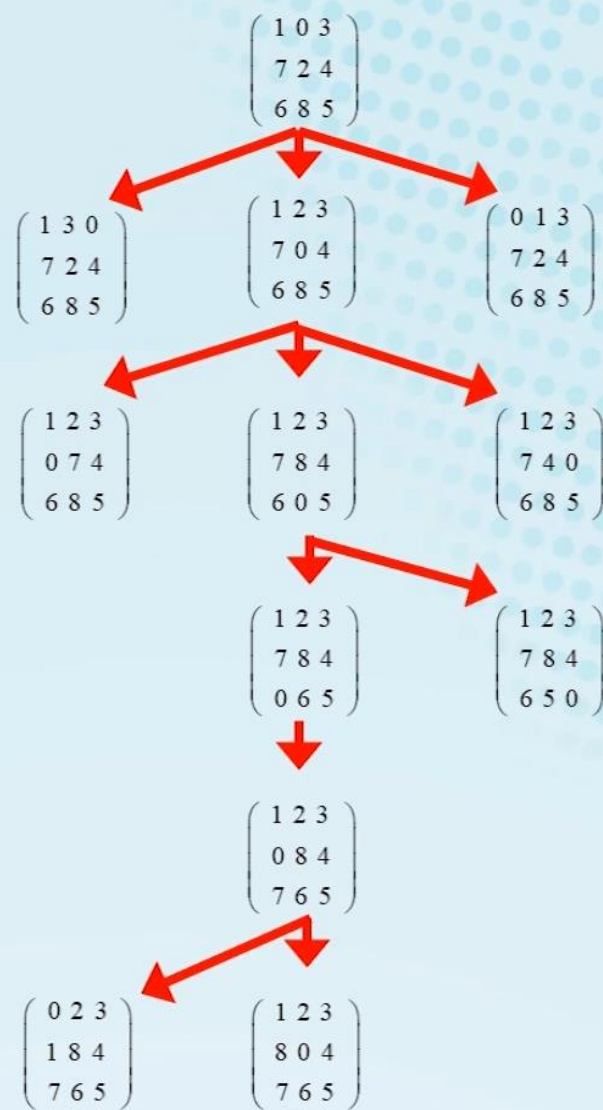


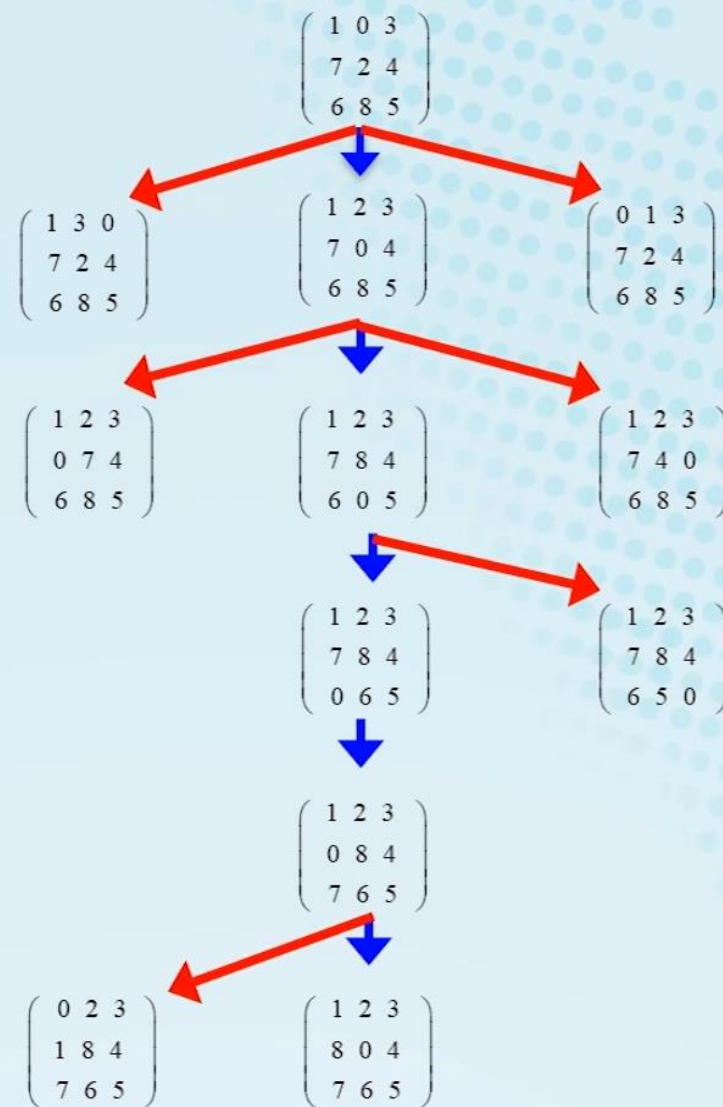
图2.2八数码难题部分状态空间图



状态图示法

状态空间的图示形式称为**状态空间图(有向图)**。其中
图的节点表示状态， 图的边表示算符

问题求解过程就是**寻求图的某一路径的问题**，实际上是一个搜索过程

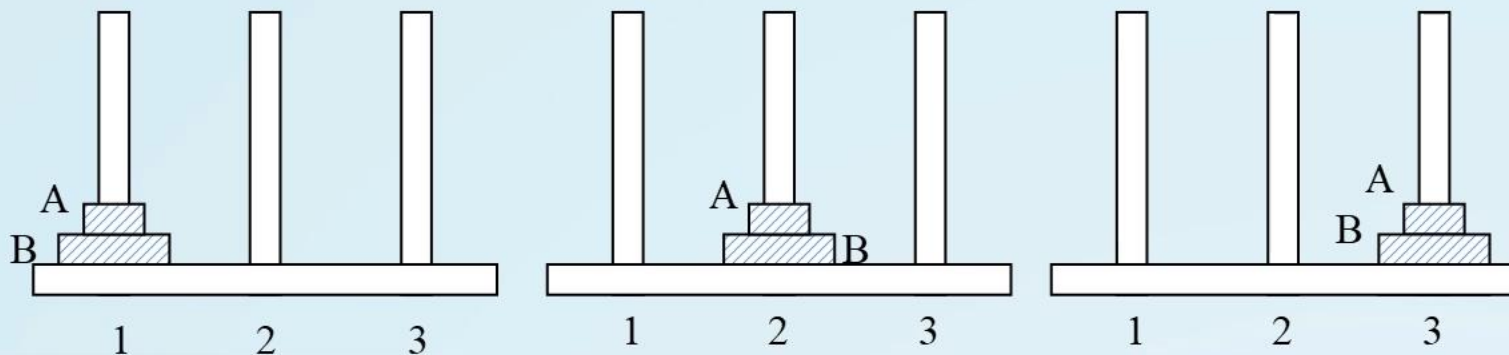




二阶梵塔难题

➤ **问题描述:** 设有三根柱子，它们的编号分别是1号、2号和3号。

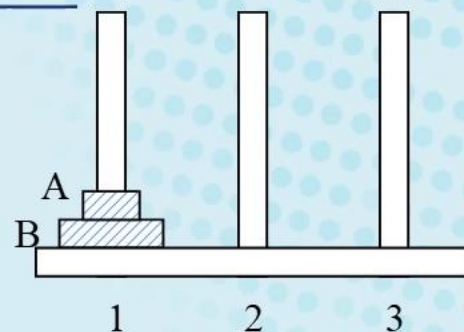
- 在初始情况下，1号柱子上穿有A、B两个盘子，A比B小，A位于B的上面。
- 要求把这两个盘子全部移到另一根柱子上，而且规定每次只能移动一个盘子，任何时刻都不能使大的位于小的上面。
- 如何将A、B移到2号或3号柱子上面？



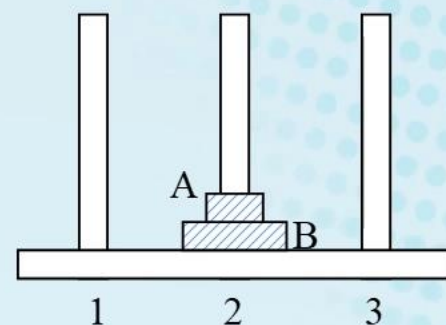


二阶梵塔难题--状态

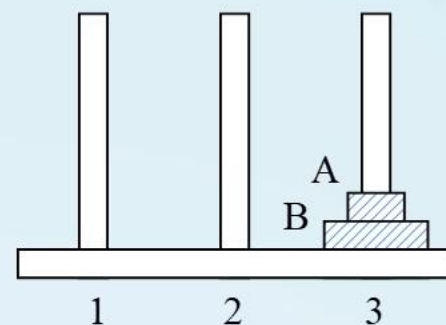
- 状态: 设 $S_K=(S_{K0}, S_{K1})$ 表示问题的状态,
 - S_{K0} 表示盘A所在的柱号, 取值1, 2, 3
 - S_{K1} 表示盘B所在的柱号, 取值1, 2, 3
- 全部可能的状态, 共有以下9种:
 - $S_0=(1,1), S_1=(1,2), S_2=(1,3),$
 - $S_3=(2,1), S_4=(2,2), S_5=(2,3),$
 - $S_6=(3,1), S_7=(3,2), S_8=(3,3).$
- 问题的初始状态集合 $S=\{S_0\}$, 目标状态集合为 $G=\{S_4, S_8\}$



$S_0=(1,1)$



$S_4=(2,2)$



$S_8=(3,3)$



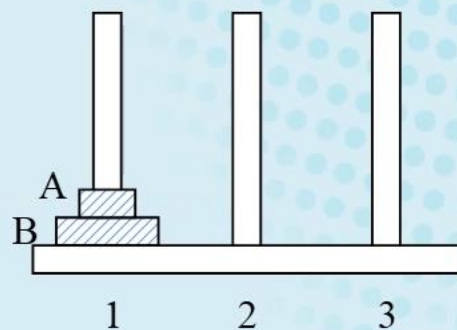
二阶梵塔难题--算符

➤ 算符: $A(i,j)$, $B(i,j)$

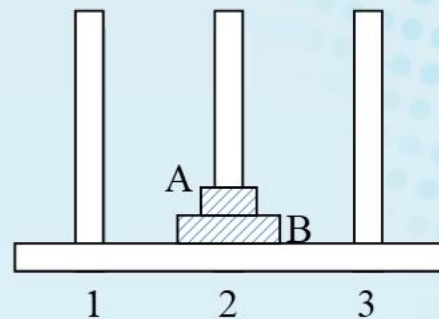
- $A(i,j)$: A盘从柱i移到柱j;
- $B(i,j)$: B盘从柱i移到柱j

➤ 全部可能的算符, 共有12种:

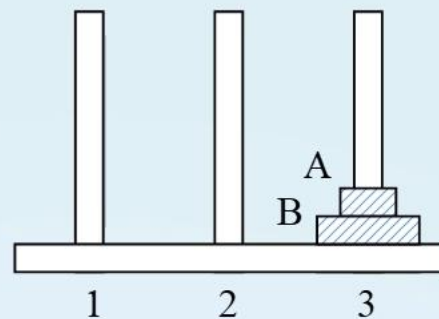
- $A(1,2)$, $A(1,3)$, $A(2,1)$, $A(2,3)$,
- $A(3,1)$, $A(3,2)$, $B(1,2)$, $B(1,3)$,
- $B(2,1)$, $B(2,3)$, $B(3,1)$, $B(3,2)$



$S_0=(1,1)$



$S_4=(2,2)$



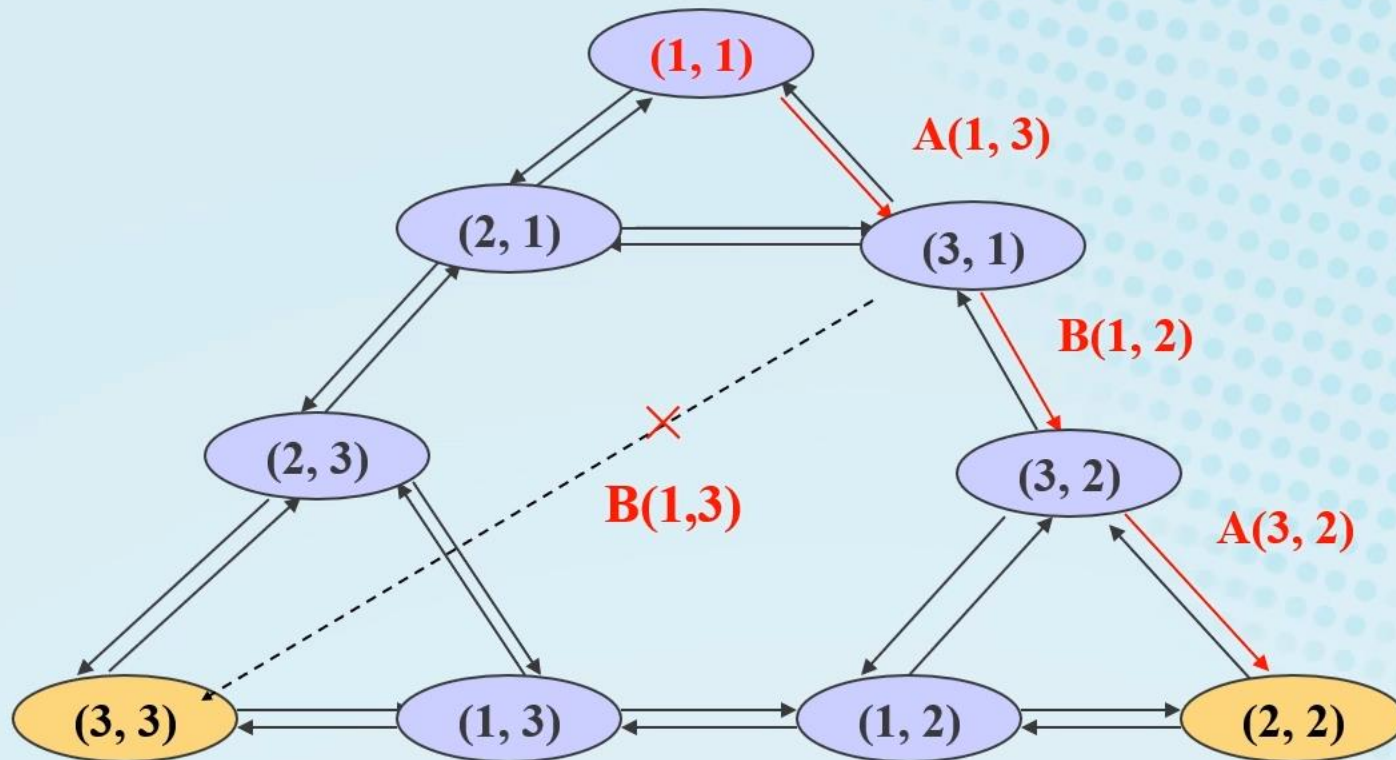
$S_8=(3,3)$



二阶梵塔难题--状态空间图

➤ 状态空间图

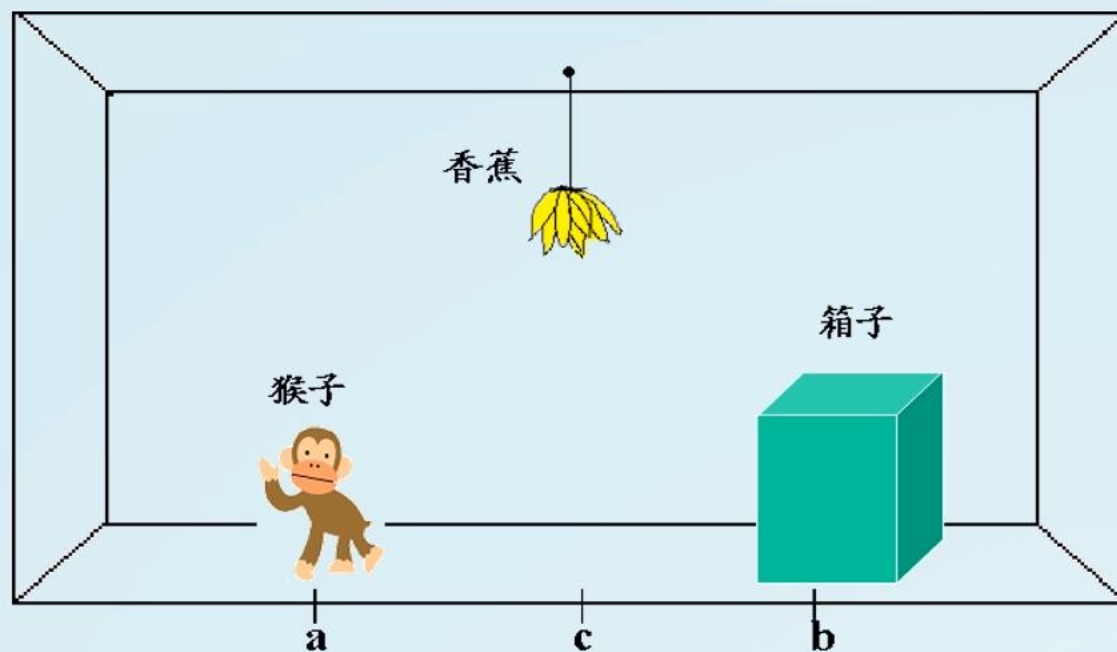
- 从初始节点(1, 1)到目标节点(2, 2)及(3, 3)的任何一条路径都是问题的一个解。
- 最短的路径长度是3，它由3个操作组成。
- 如从(1, 1)开始，通过使用操作A(1, 3)、B(1, 2)及A(3, 2)，可到达(2, 2)。



2.3 二阶梵塔难题的状态空间图



- 问题描述：在一个房间内有一只猴子，一个箱子和一束香蕉。香蕉挂在天花板下方，但猴子的高度不足以碰到它。那么这只猴子怎样才能摘到香蕉呢？



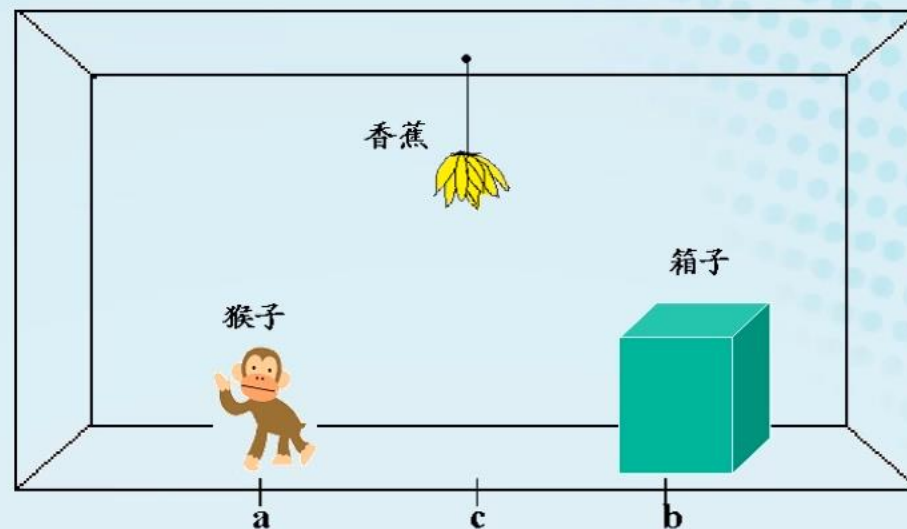


➤ 用一个四元表列 (W, Y, x, z) 表示这个问题状态

- W : 猴子的水平位置
- Y : 箱子的水平位置
- x : 当猴子在箱子顶上时取 $x=1$; 否则取 $x=0$
- z : 当猴子摘到香蕉时取 $z=1$; 否则取 $z=0$

➤ 初始和目标状态为:

- 初始状态: $(a, b, 0, 0)$
- 目标状态: $(c, c, 1, 1)$





- **goto(U)**表示猴子走到水平位置U，即：

$$(W, Y, 0, 0) \xRightarrow{\text{goto}(U)} (U, Y, 0, z)$$

- **pushbox(V)**表示把箱子推到水平位置V，即：

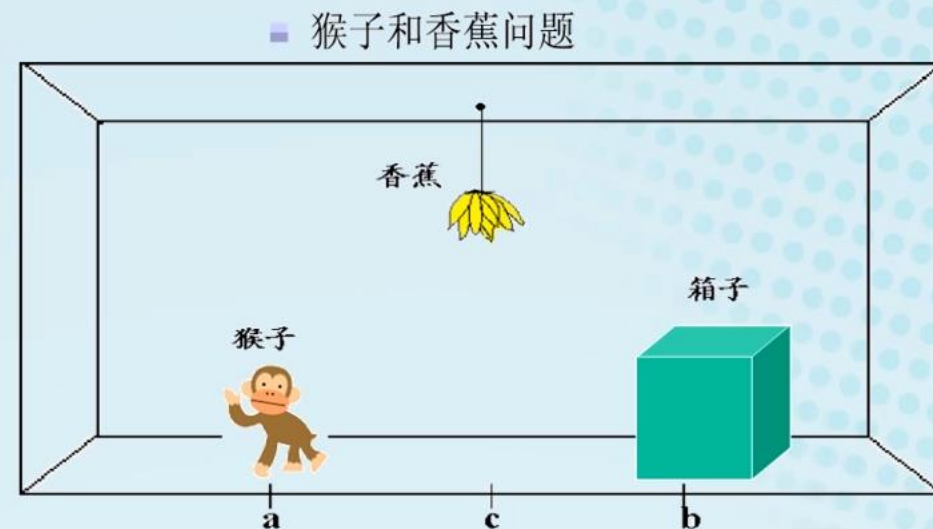
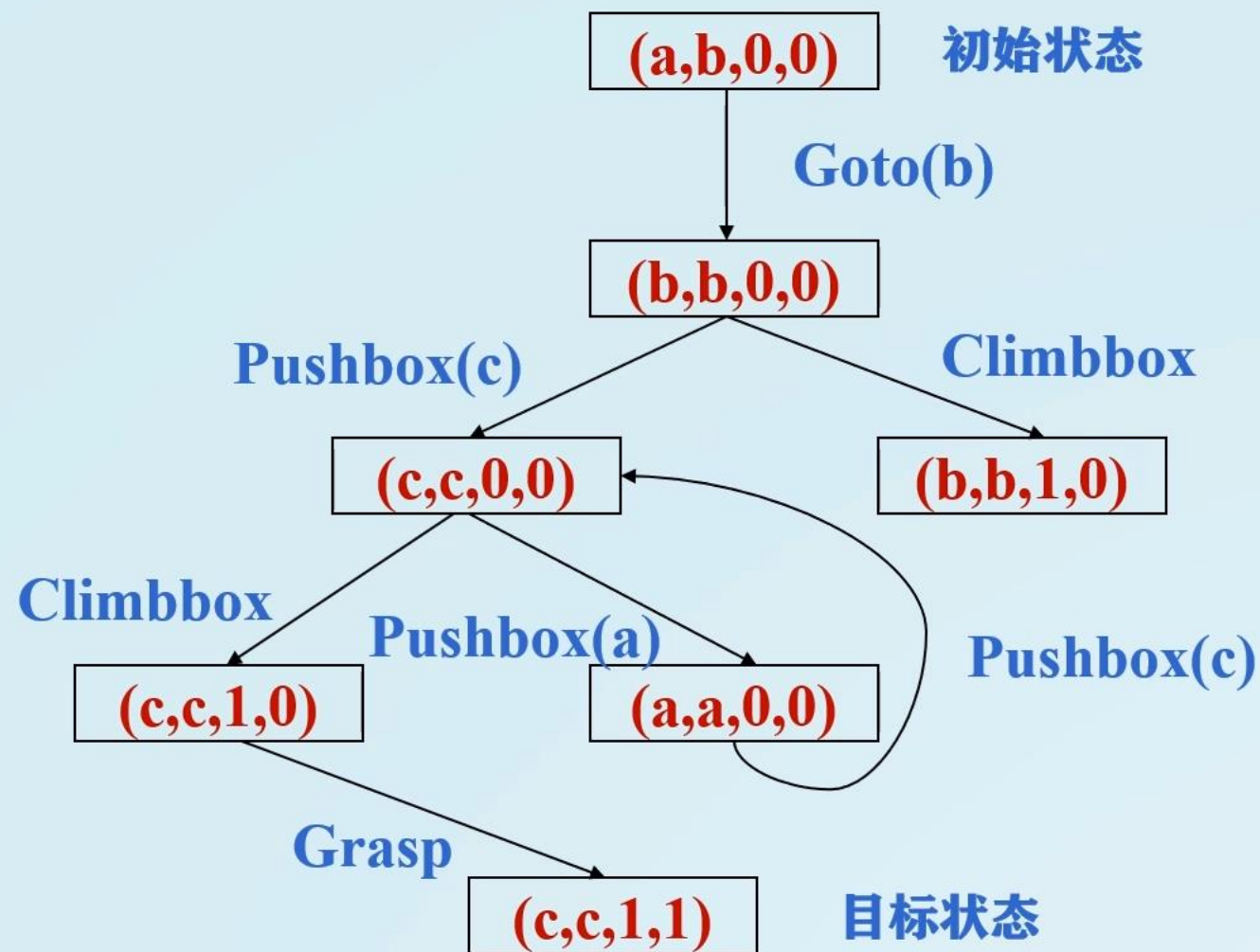
$$(W, W, 0, z) \xRightarrow{\text{pushbox}(V)} (V, V, 0, z)$$

- **Climbbox**：猴子爬上箱顶，即：

$$(W, W, 0, z) \xRightarrow{\text{climbbox}} (W, W, 1, z)$$

- **Grasp**：猴子摘到香蕉，即：

$$(c, c, 1, 0) \xRightarrow{\text{grasp}} (c, c, 1, 1)$$



解序列为: {Goto(b), Pushbox(c), Climbbox, Grasp}



小结





西安电子科技大学
XIDIAN UNIVERSITY



人工智能导论

图搜索策略

主讲人：刘若辰

西安电子科技大学 人工智能学院



引言

- 问题表示是问题求解所必须的，从问题表示到问题的解决，有一个求解过程,也就是搜索过程。
- 状态空间法的求解过程等价于在状态空间图中搜索一条从初始状态到目标状态的路径问题。

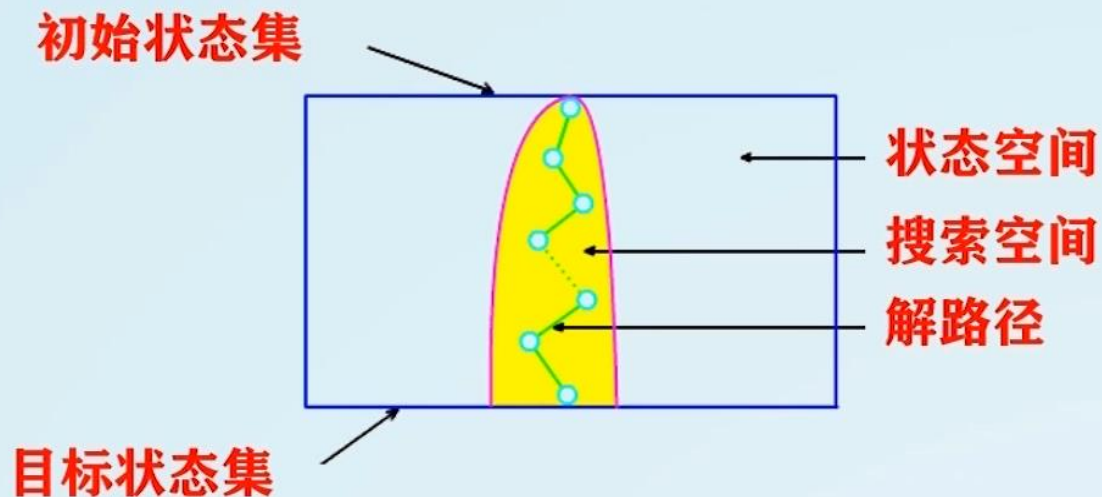


图2.4：状态空间法



状态图搜索

- 下图是一个9格子迷宫，如果我们把该迷宫的每一个格子以及入口和出口都作为节点，把通道作为边,则该迷宫状态空间图如图2.5所示。
- 走迷宫其实就是从该有向图的初始节点(入口)出发, **寻找通向目标节点(出口)的路径的问题。**

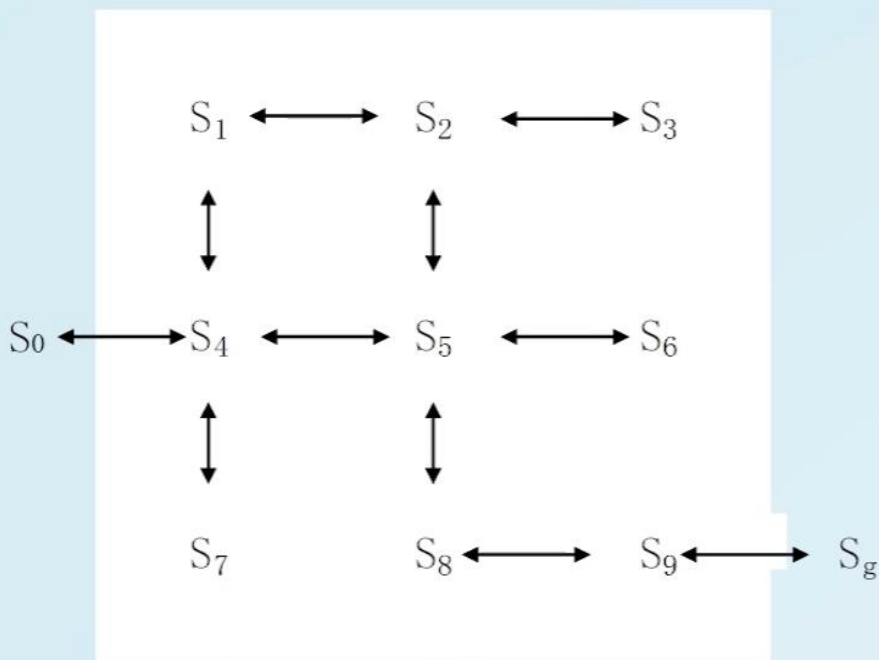
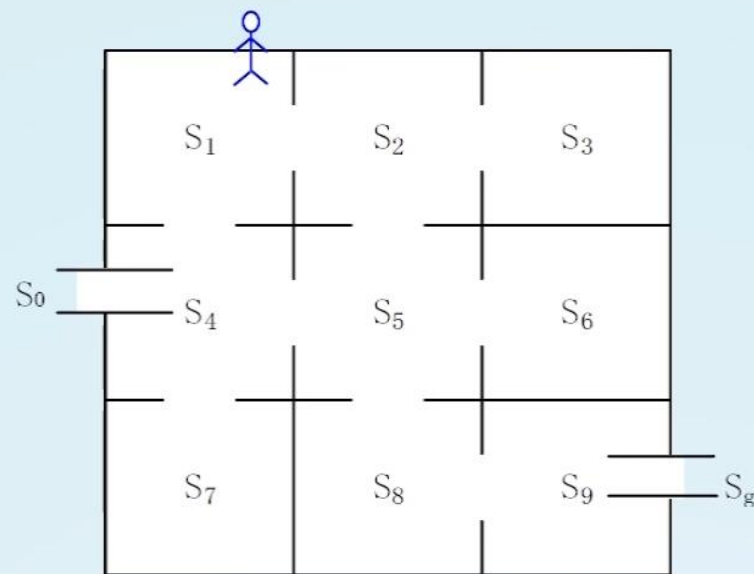
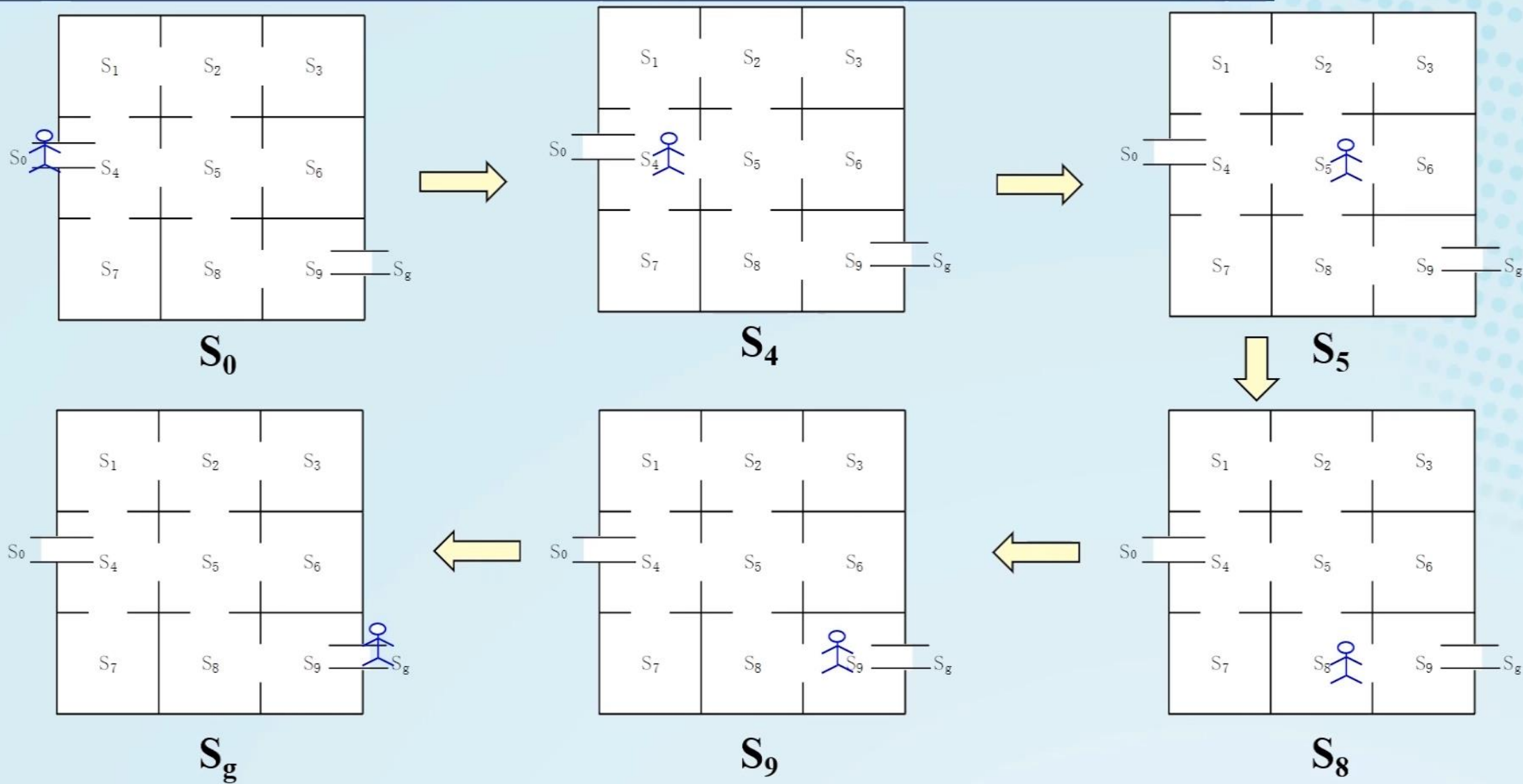


图2.5: 迷宫的状态空间图



9格子迷宫



$S_0 \rightarrow S_4 \rightarrow S_5 \rightarrow S_8 \rightarrow S_9 \rightarrow S_g$



状态图搜索

由于搜索的目的是为了寻找初始节点到目标节点的路径，所以在搜索过程中就得随时记录搜索轨迹。

- 必须记住下一步还可以走哪些点

OPEN表(记录待扩展的节点)

- 必须记住哪些点走过了

CLOSED表(记录已扩展的节点)

- 必须记住从目标返回的路径

每个表示状态的节点结构中必须有指向父节点的指针

所谓对一个节点进行“扩展”是指对该节点用某个可用操作进行作用，生成该节点的一组子节点（后继节点）



状态图搜索

登记当前待
考察的节点

记录考察过的
节点

OPEN表与CLOSED表

OPEN

节点	父节点编号

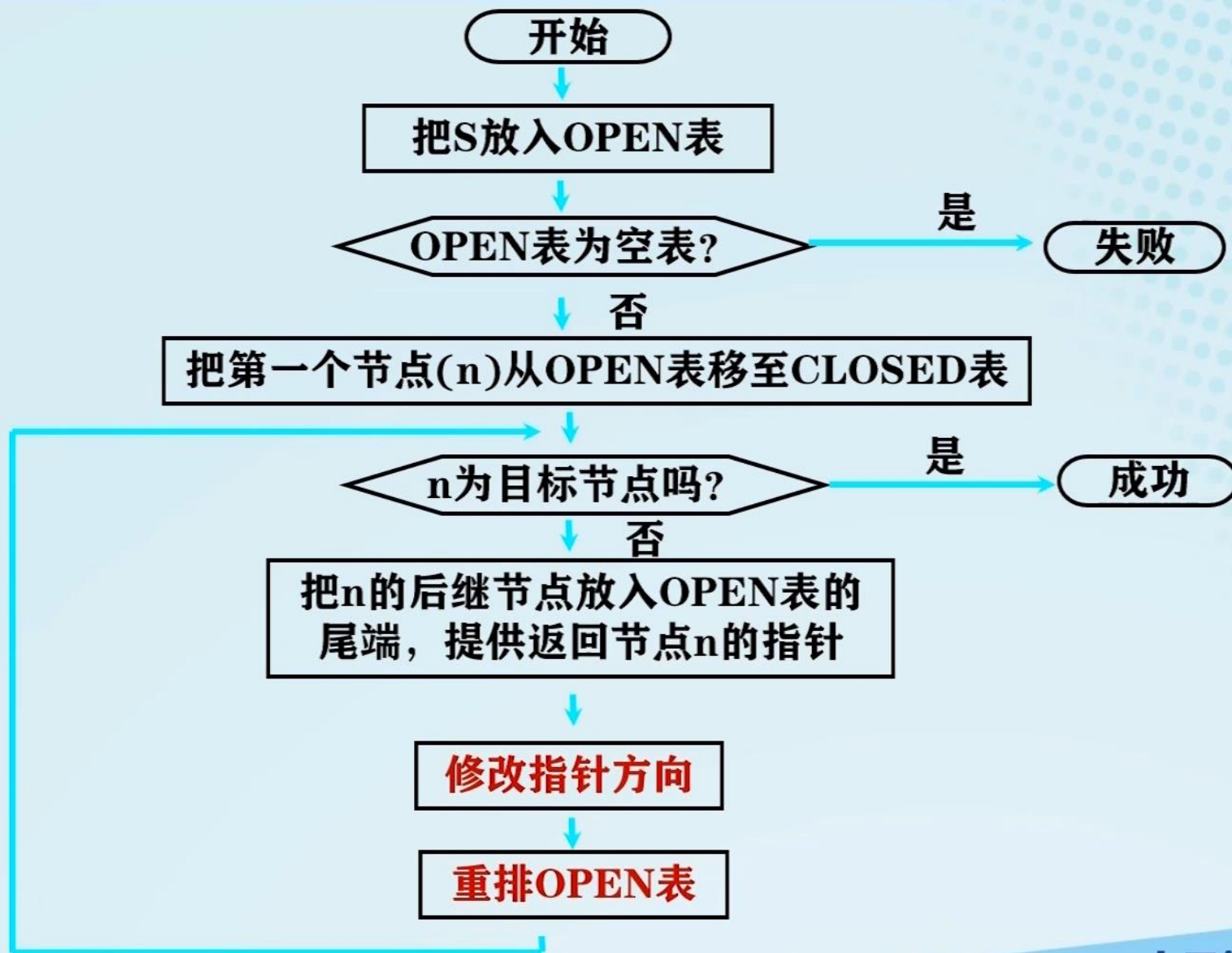
CLOSED

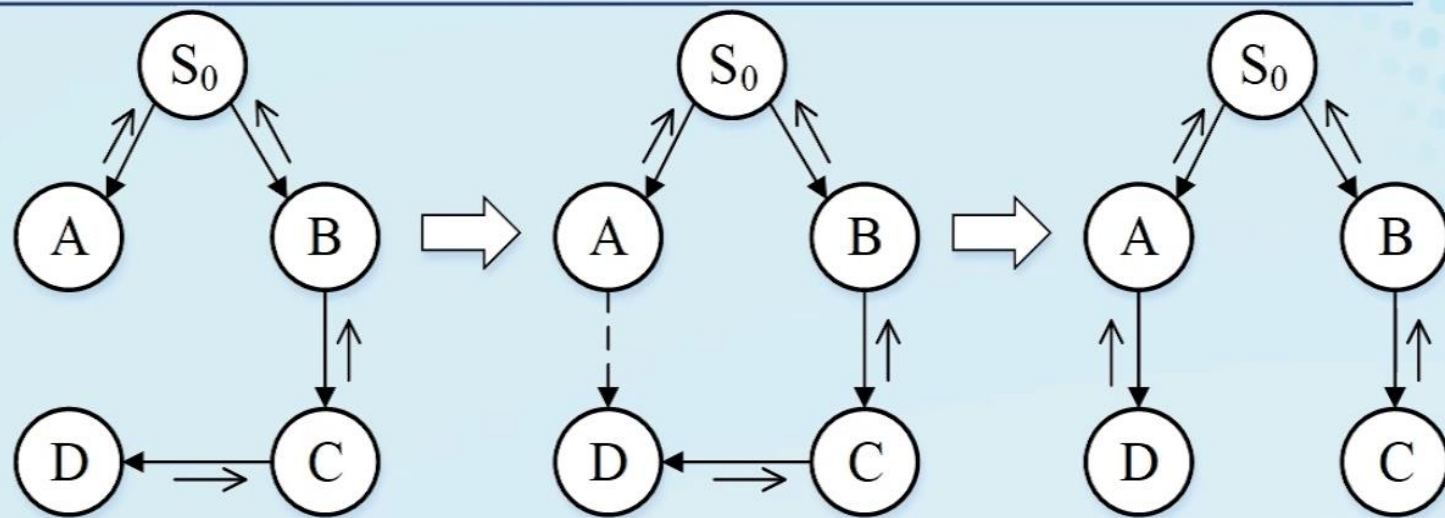
编号	节点	父节点编号



图搜索过程

图搜索流程图





修改返回指针示例

进一步说明:

- (1)第7步中修改返回指针的原因是, 因为这些节点又被第二次生成, 所以它们返回初始节点的路径已有两条, 但这两条路径的“长度”可能不同。那么, 当新路短时自然要走新路。
- (2)这里对路径的长短是按路径上的节点数来衡量的, 后面我们将会看到路径的长短也可以其“代价”(如距离、费用、时间等)衡量。



图搜索过程

- 注：这是一个通用的搜索过程，后面讨论的状态空间各种搜索策略都是其特例。
- 算法结束后，将生成一个图 G ，称为搜索图。同时由于每个节点都有一个指针指向父节点，这些指针指向的节点构成 G 的一个支撑树，称为搜索树。
- 从目标节点开始，将指针指向的状态回串起来，即找到一条解路径。



状态图搜索--例题一

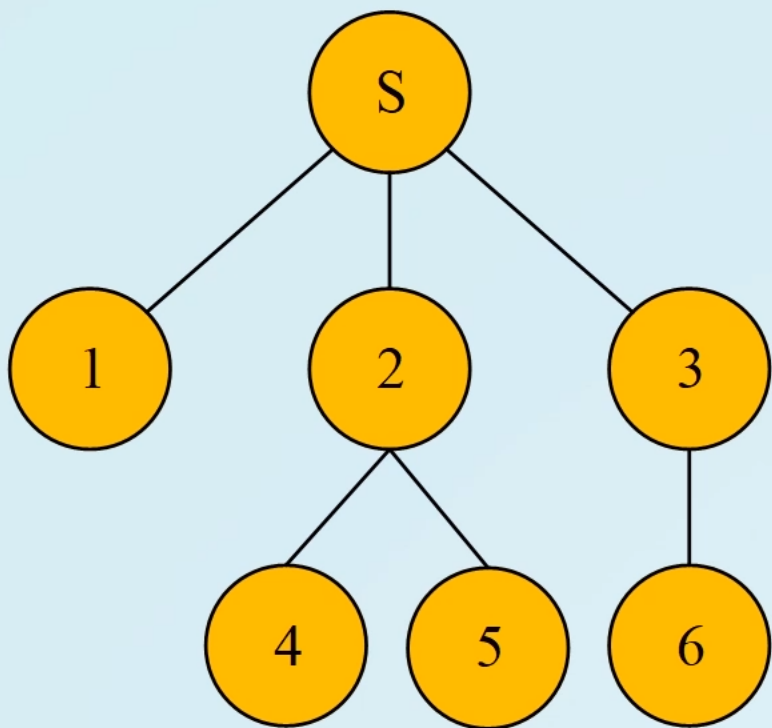


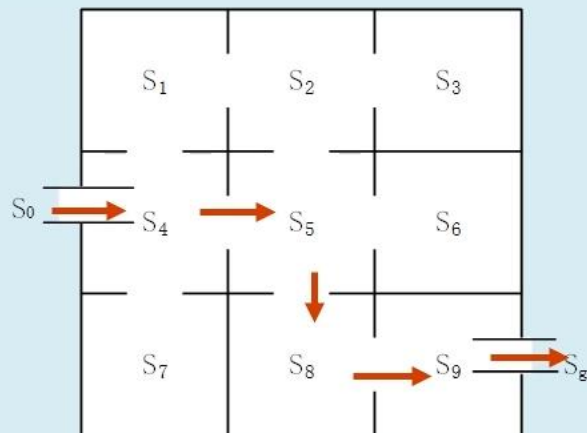
图2.6：一个例子

Open 表	Close 表
{S}	{}
{}	{S}
{1,2,3}	{S}
{2,1,3}	{S}
{1,3}	{S,2}
{1,3,4,5}	{S,2}
{3,1,4,5}	{S,2}
{1,4,5}	{S,2,3}
{1,4,5,6}	{S,2,3}



状态图搜索--例题二

➤ 9格子迷宫:解法一



$$S_0 \rightarrow S_4 \rightarrow S_5 \rightarrow S_8 \rightarrow S_9 \rightarrow S_g$$

Open表

Closed 表

{S₀}

{S₄}

{S₅,S₁,S₇}

{S₅,S₁,S₇}

{S₁,S₇}

{S₁,S₇,S₈,S₆,S₂}

{S₈,S₆,S₂,S₁,S₇}

{S₆,S₂,S₁,S₇,S₉}

{S₉,S₆,S₂,S₁,S₇}

{S₆,S₂,S₁,S₇,S_g}

{S_g,S₆,S₂,S₁,S₇}

{S₆,S₂,S₁,S₇}

{S₀}

{S₀,S₄}

{S₀,S₄}

{S₀,S₄,S₅}

{S₀,S₄,S₅}

{S₀,S₄,S₅}

{S₀,S₄,S₅,S₈}

{S₀,S₄,S₅,S₈}

{S₀,S₄,S₅,S₈,S₉}

{S₀,S₄,S₅,S₈,S₉}

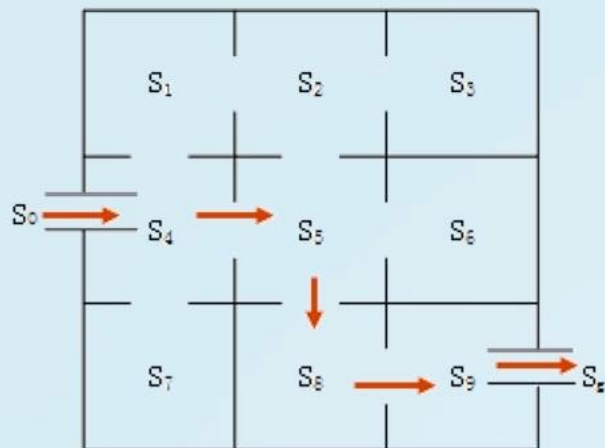
{S₀,S₄,S₅,S₈,S₉,S_g}



状态图搜索--例题二

➤ 9格子迷宫:解法二

$$S_0 \rightarrow S_4 \rightarrow S_5 \rightarrow S_8 \rightarrow S_9 \rightarrow S_g$$



$$S_0 \rightarrow S_4 \rightarrow S_5 \rightarrow S_8 \rightarrow S_9 \rightarrow S_g$$

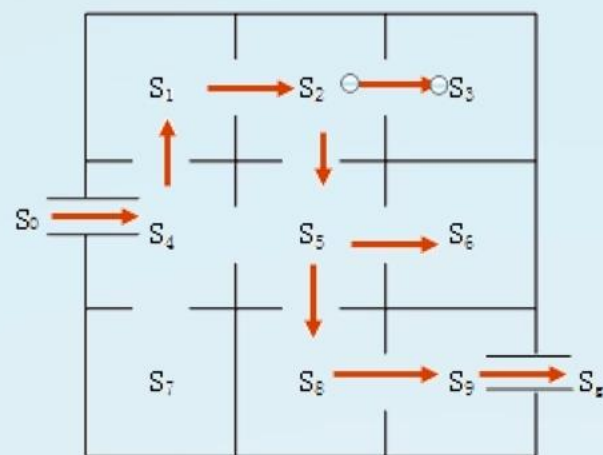


图2.7：九格子迷宫问题的另一种解法



图搜索分类

- 对OPEN表中节点排序方式产生了不同的搜索策略，不同的搜索策略效率不同。
- 这种排序可以是任意的即盲目的(属于盲目搜索)，也可以用各种启发思想或其它准则为依据(属于启发式搜索)。

- 无信息搜索（盲目式搜索）

- 宽度优先搜索

- 深度优先搜索

- 等代价搜索

- 有信息搜索（启发式搜索）

- A算法

- A*算法

➤ 图搜索策略



西安电子科技大学
XIDIAN UNIVERSITY



人工智能导论

盲目式搜索

主讲人：刘若辰

西安电子科技大学 人工智能学院



盲目式搜索

➤ 概念

- 按预定的控制策略进行搜索，在搜索过程中获得的中间信息不用来改进控制策略
(**没有启发信息的一种搜索形式**)
- 一般只适用于求解比较简单的问题

➤ 种类

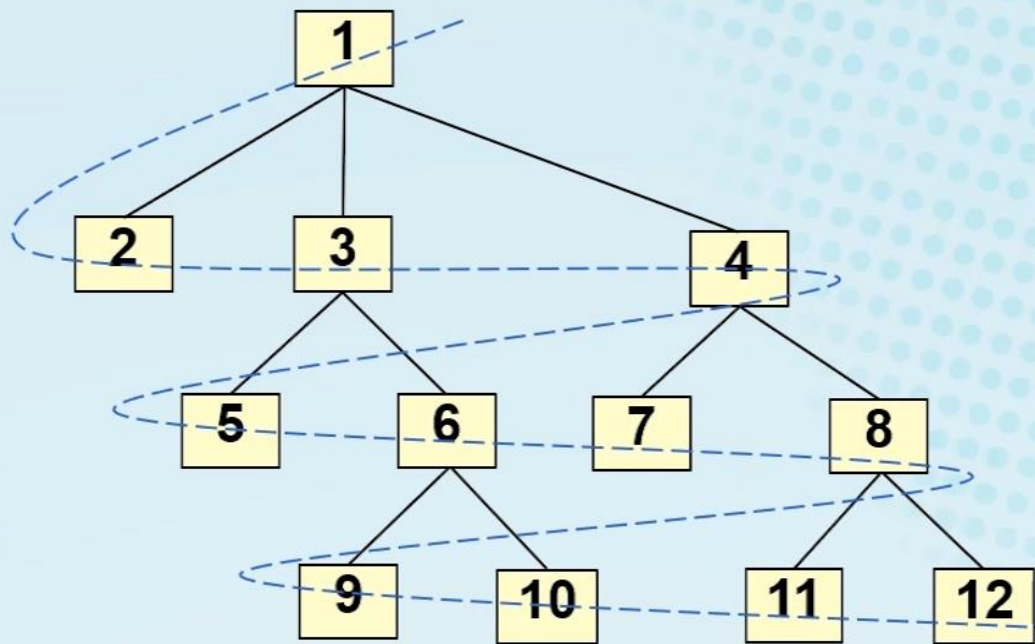
- **宽度优先**
- **深度优先**
- **等代价搜索**



宽度优先搜索

➤ 宽度/广度优先搜索基本思想:

- 首先扩展初始节点;
- 接着扩展初始节点的所有后继节点;
- 然后再扩展后继节点的后继, 依此类推;
- 在第 n 层节点还没有全部搜索完之前, 不进入第 $n+1$ 层节点的搜索。

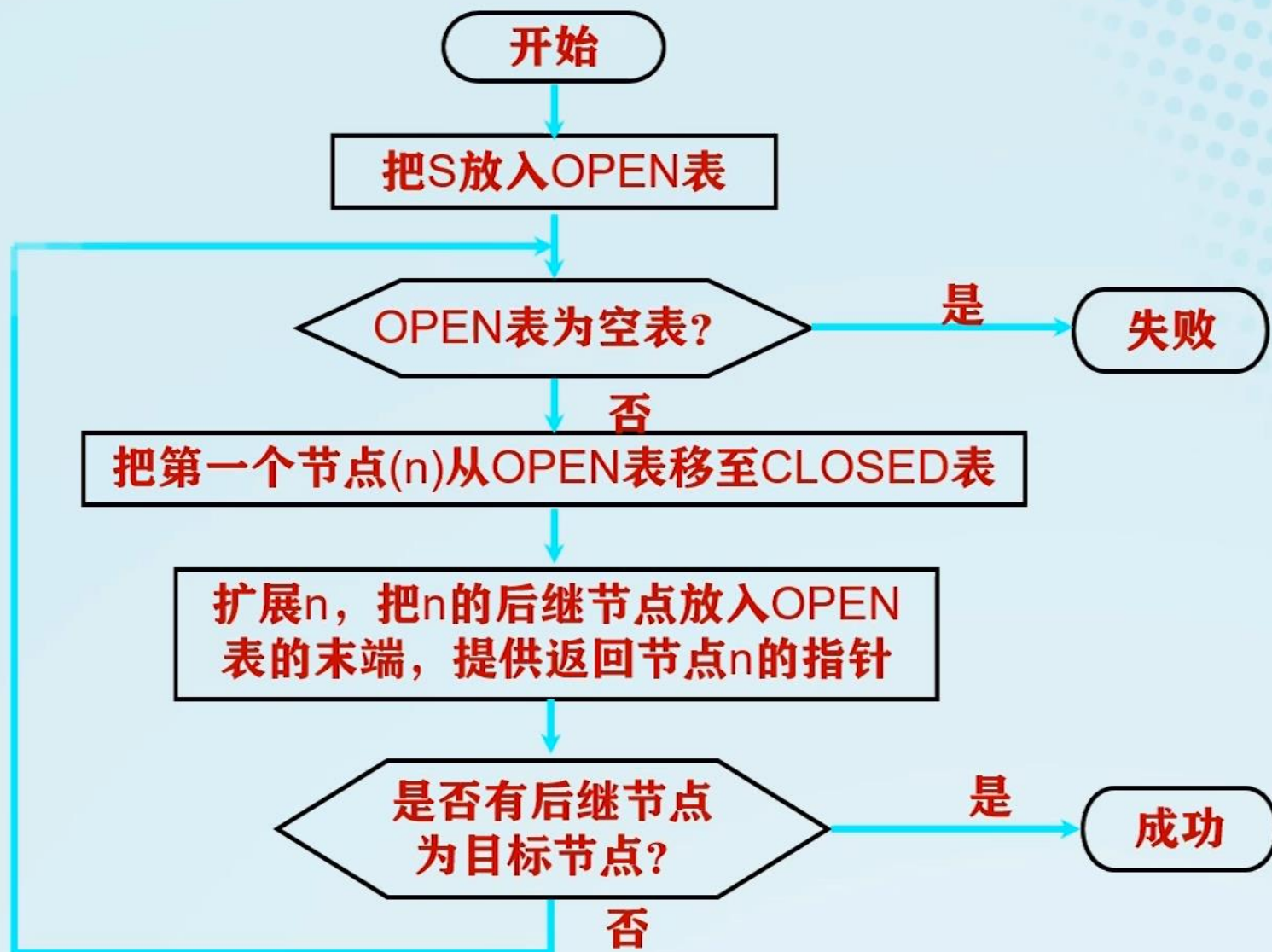


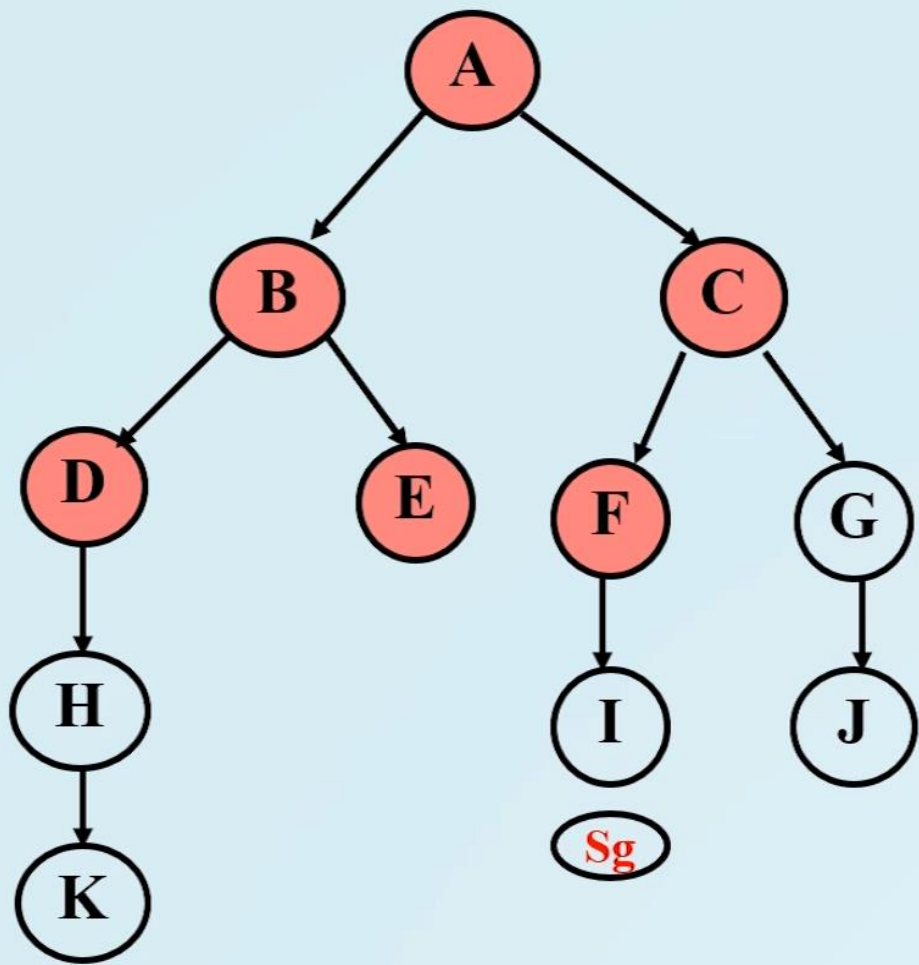
先生成的节点先扩展!



宽度优先搜索

宽度优先搜索流程图



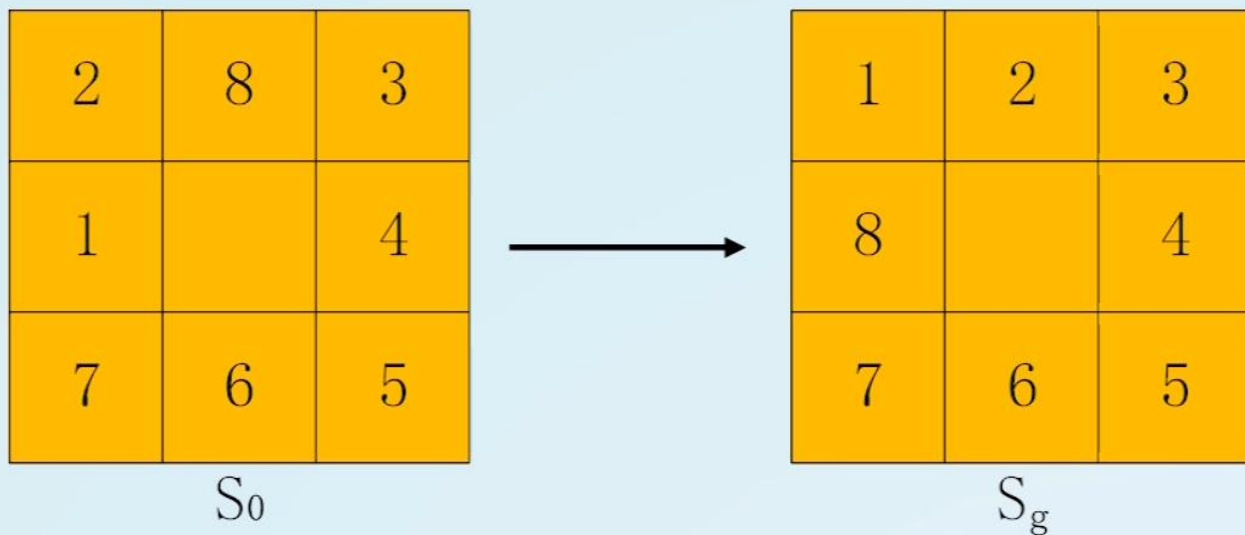


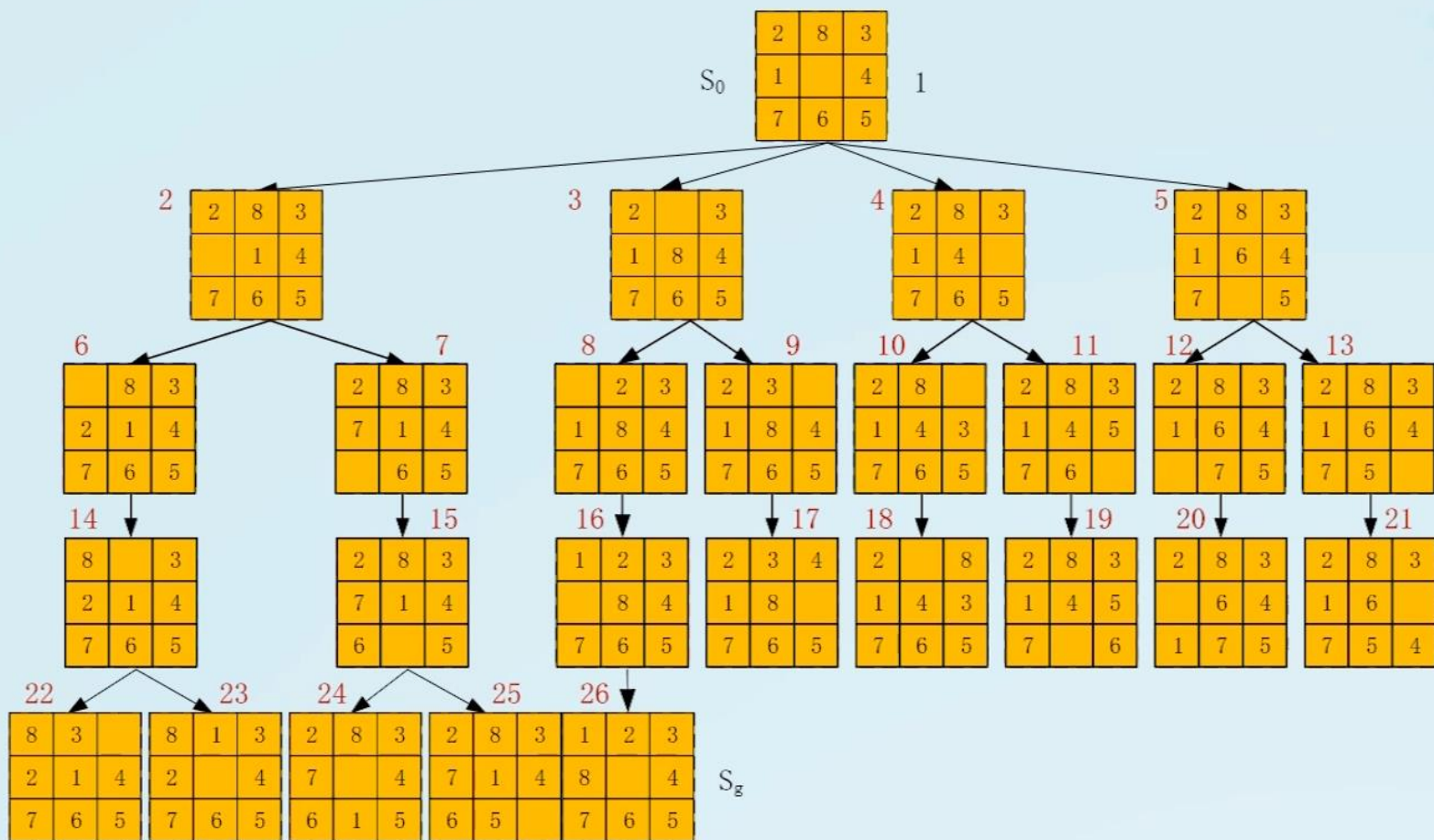
Open 表	Close 表
{A}	{}
{B,C}	{A}
{C,D,E}	{A,B}
{D,E,F,G}	{A,B,C}
{E,F,G,H}	{A,B,C,D}
{F,G,H}	{A,B,C,D,E}
{G,H,I}	{A,B,C,D,E,F}



八数码难题

在 3×3 的棋盘，摆有八个棋子，每个棋子上标有1至8的某一数字。棋盘上还有一个空格，与空格相邻的棋子可以移到空格中。





S_0
↓
3
↓
8
↓
16
↓
26



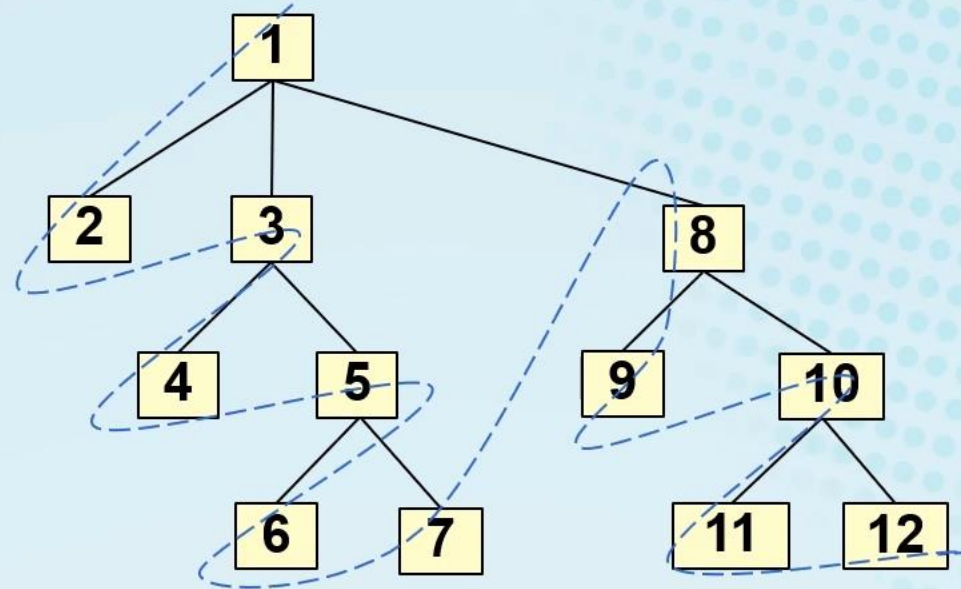
宽度优先搜索的性质

- 属于图搜索方法
- 新扩展的节点排在open表的末端
- 当问题有解时，一定能找到解
- 方法与问题无关，具有通用性
- 效率较低



➤ 深度优先搜索基本思想:

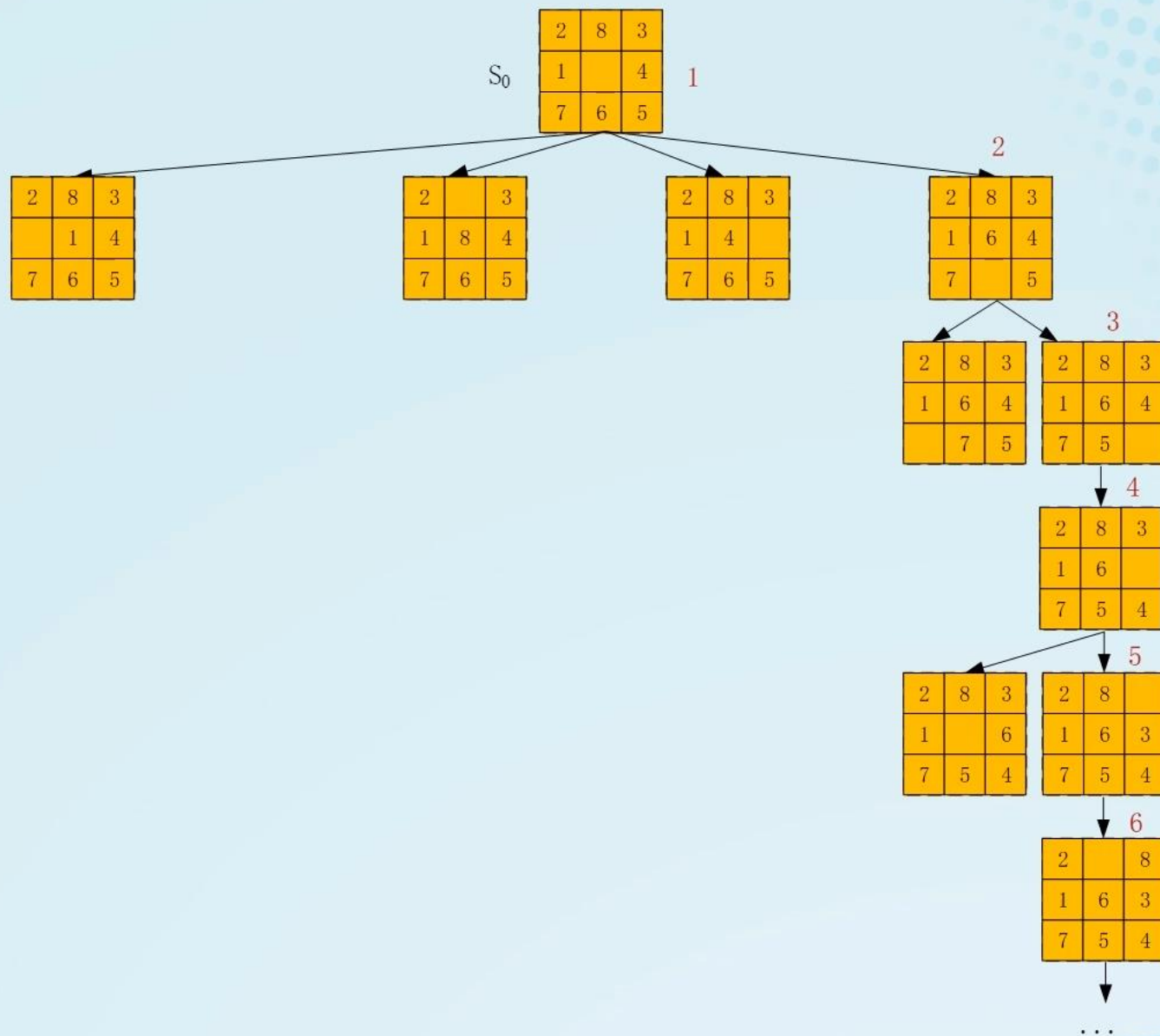
- 总是扩展搜索树的当前扩展分支中最深的节点;
- 搜索直接伸展到搜索树的最深层, 直到那里的节点没有后继节点;
- 然后搜索算法回退到 下一个还有未扩展后继节点的上层节点继续扩展。



先扩展最新产生的(即最深的)节点



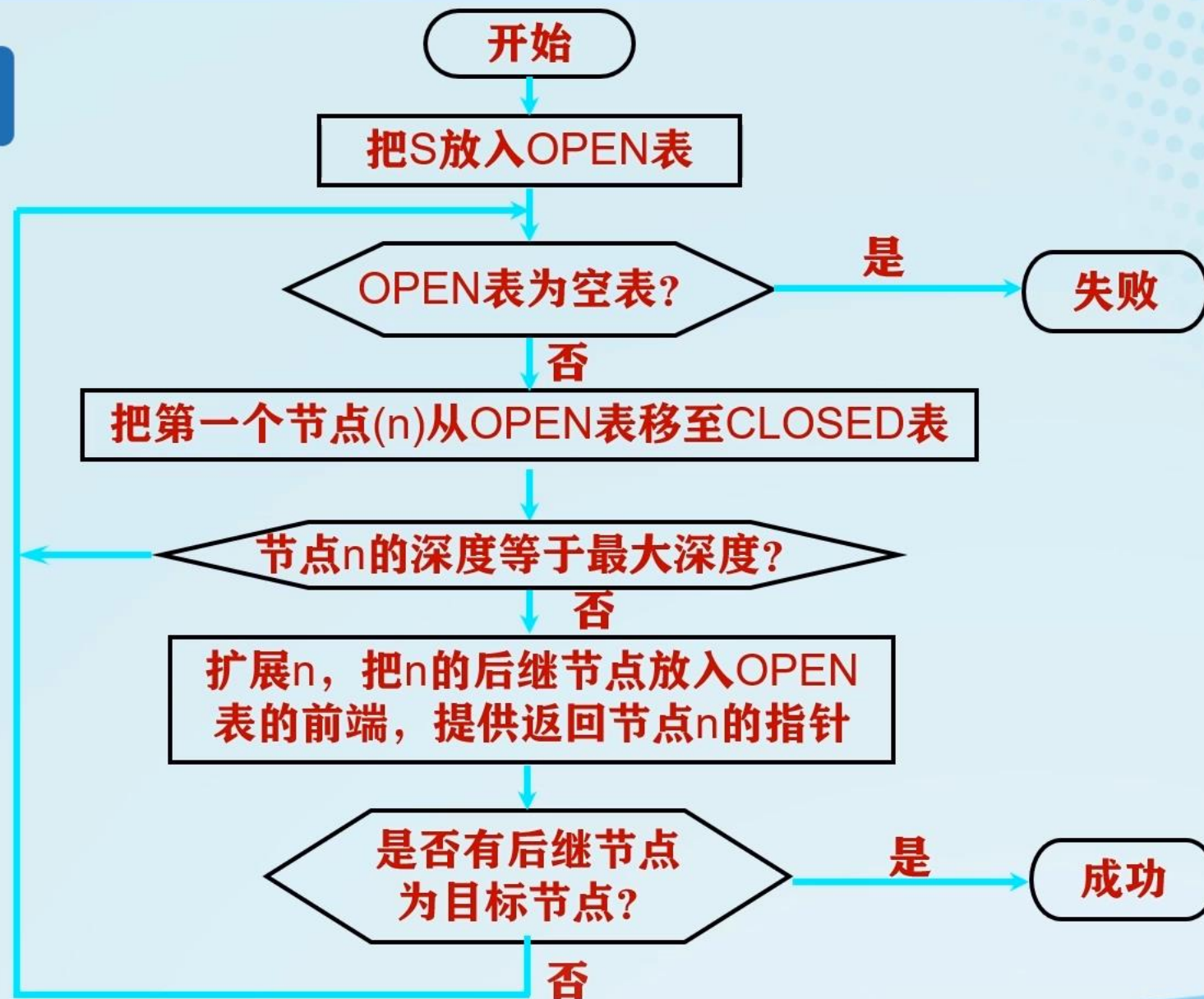
深度优先搜索

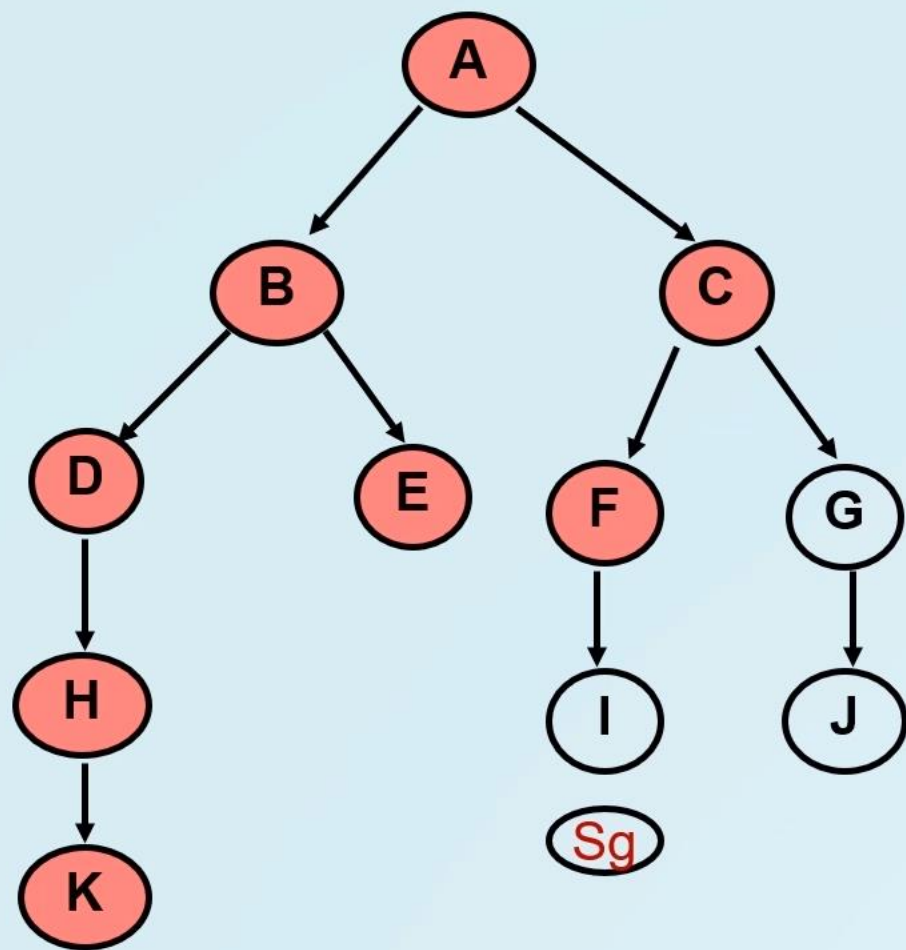




深度优先搜索

深度优先搜索流程图





Open 表	Close 表
{A}	{}
{B,C}	{A}
{D,E,C}	{A,B}
{H,E,C}	{A,B,D}
{K,E,C}	{A,B,D,H}
{E,C}	{A,B,D,H,K}
{C}	{A,B,D,H,K,E}
{F,G}	{A,B,D,H,K,E,C}
{I,G}	{A,B,D,H,K,E,C,F}



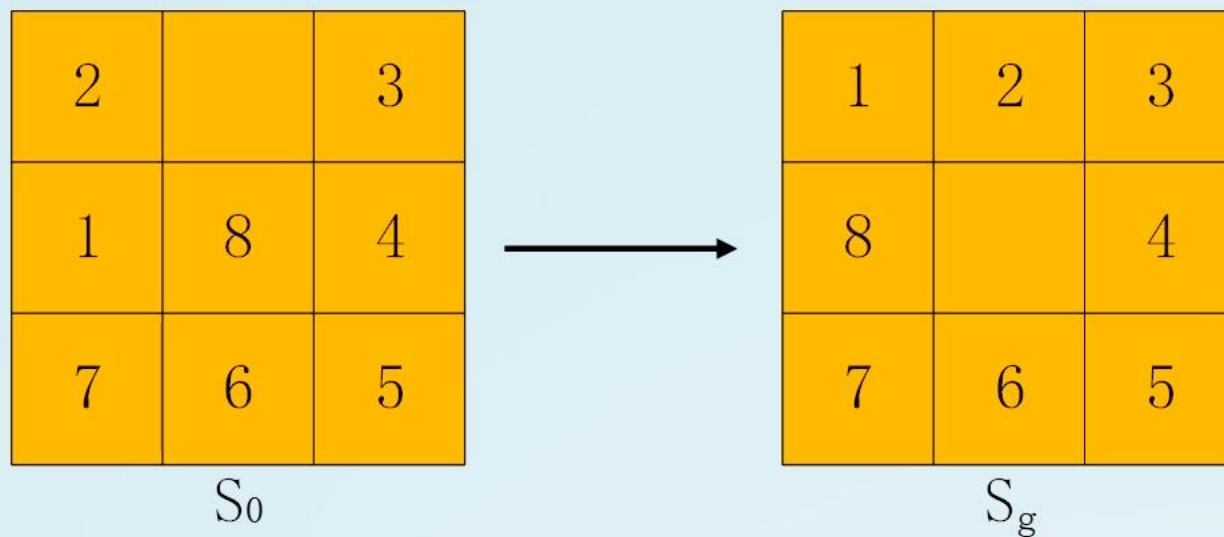
深度优先搜索

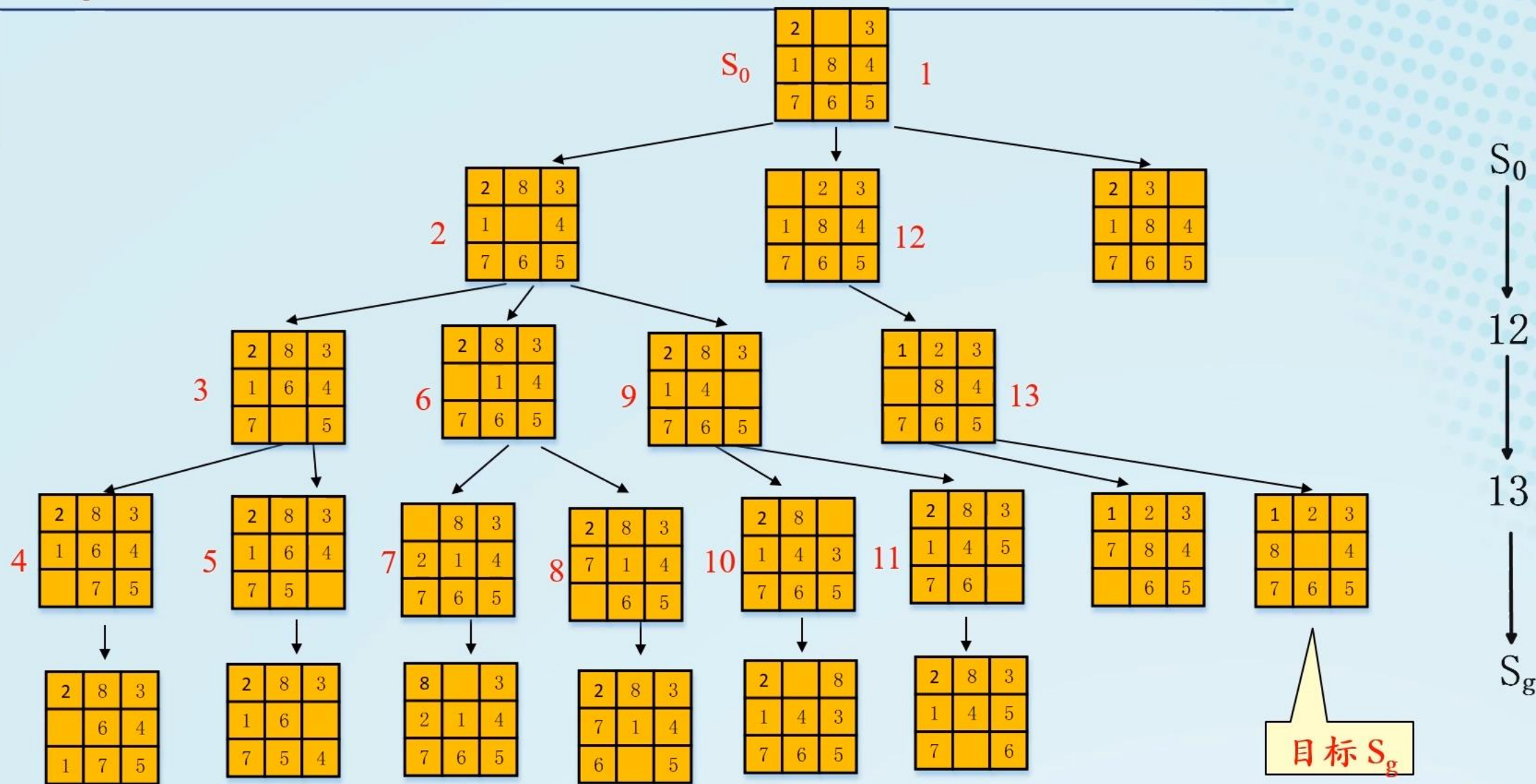
- 防止搜索过程沿着无益的路径扩展下去，往往给出一个节点扩展最大深度 —— 深度界限。
- 定义-节点的深度:
 - (1) 起始节点的深度为0。
 - (2) 任何其他节点的深度等于其父辈节点的深度加1。



➤ 八数码难题

在 3×3 的棋盘，摆有八个棋子，每个棋子上标有1至8的某一数字。棋盘上还有一个空格，与空格相邻的棋子可以移到空格中。

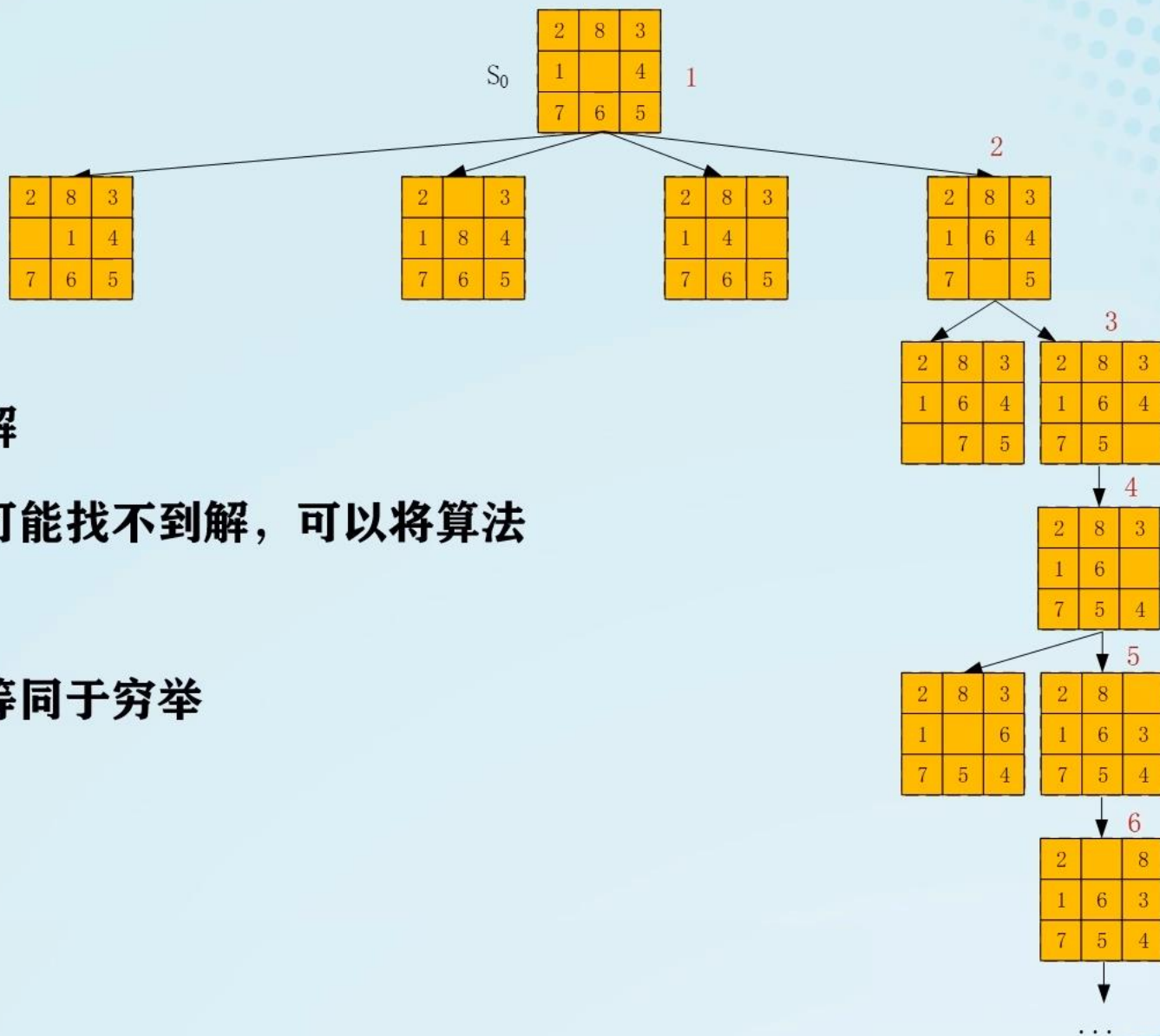




八数码难题的深度优先搜索（深度界限：4）



深度优先搜索



- 一般不能保证找到最优解
- 当深度限制不合理时，可能找不到解，可以将算法改为可变深度限制
- 最坏情况时，搜索空间等同于穷举



等代价搜索

➤ 等代价搜索基本思想:

- 等代价搜索是宽度优先搜索的一种推广，不是沿着等长度路径断层进行扩展，而是沿着等代价路径断层进行扩展。
- 状态图中每条连接弧线上的有关代价，表示时间、距离等花费。
- 节点代价的定义：

$g(n)$ ：表示从初始节点 S_0 到节点 n 的代价；

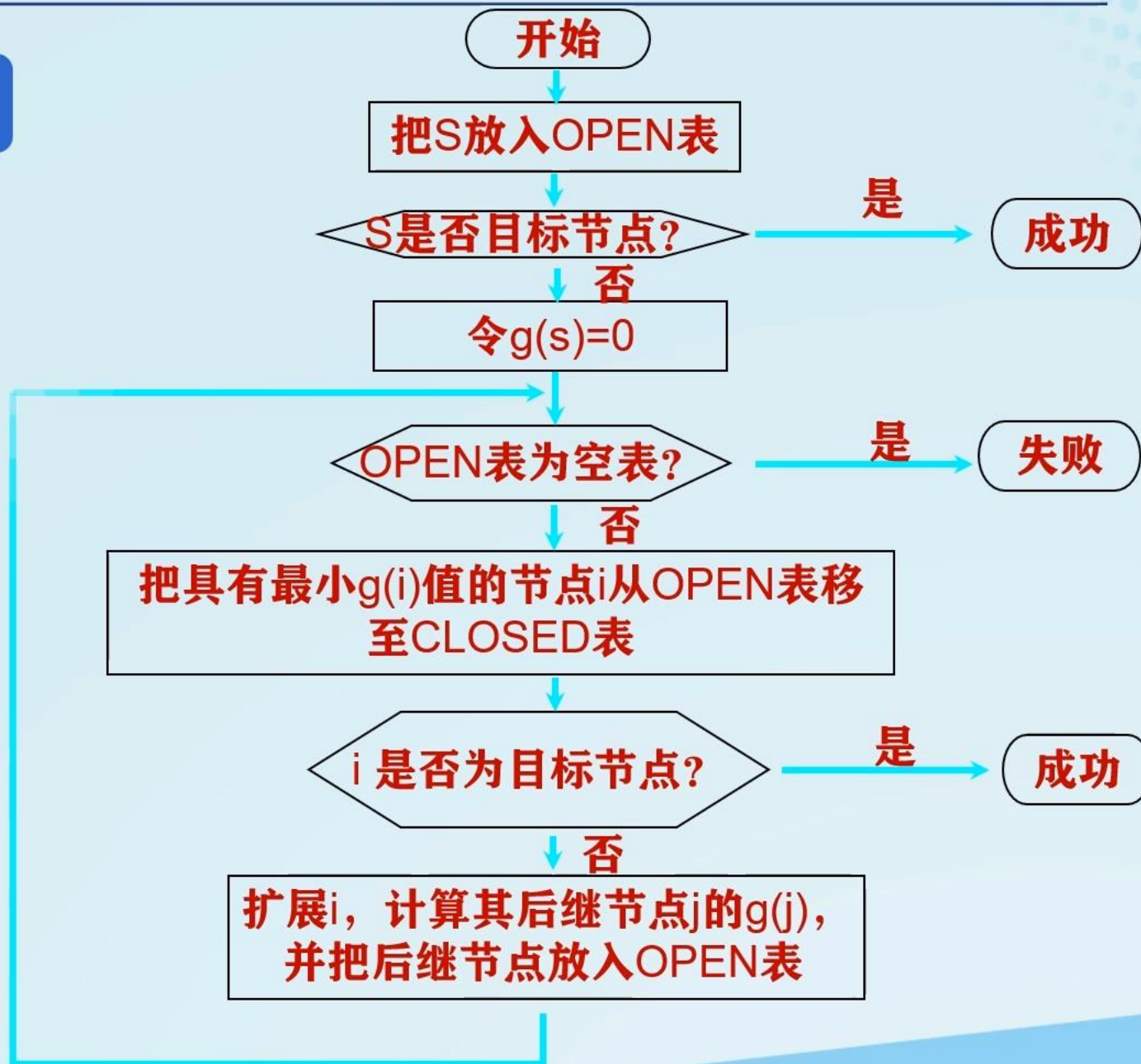
$c(n_1, n_2)$ ：表示从父节点 n_1 到其子节点 n_2 的代价；

$$g(n_2) = g(n_1) + c(n_1, n_2)$$



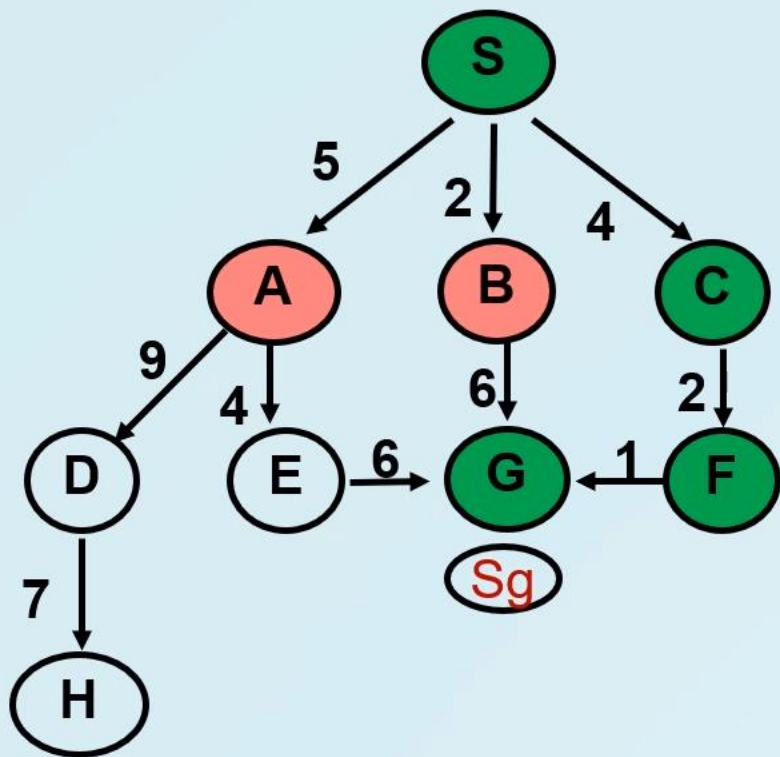
等代价搜索

等代价搜索流程图





等代价搜索



Open 表	Close 表
{S(0)}	{}
{B:2,C:4,A:5}	{S:0}
{C:4,A:5,G:(2+6)}	{S:0,B:2}
{A:5,F:(4+2),G:(2+6)}	{S:0,B:2,C:4}
{F(4+2),G(2+6),E(5+4),D:(5+9)}	{S:0,B:2,C:4, A:5}
{G: (4+2+1),G: (2+6),E: (5+4),D:(5+9)}	{S:0,B:2,C:4, A:5, F: (4+2)}
{G: (2+6),E: (5+4),D:(5+9)}	{S:0,B:2,C:4, A:5, F: (4+2), G: (4+2+1)}



盲目式搜索小结

- 宽度优先搜索是按“层”进行搜索的，先进入OPEN表的节点先被考察；
- 深度优先搜索是沿着纵深方向进行搜索的，后进入OPEN表的节点先被考察；
- 等代价搜索首先扩展最小代价节点。

按预先设置的策略或已付出的代价进行搜索，没有利用搜索过程中的信息指导搜索!



西安电子科技大学
XIDIAN UNIVERSITY



人工智能导论

启发式搜索（上）

主讲人：刘若辰

西安电子科技大学 人工智能学院



启发式搜索

1. A算法
2. A*算法

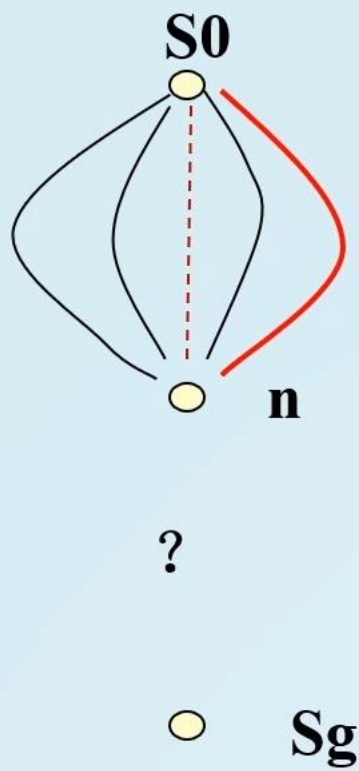




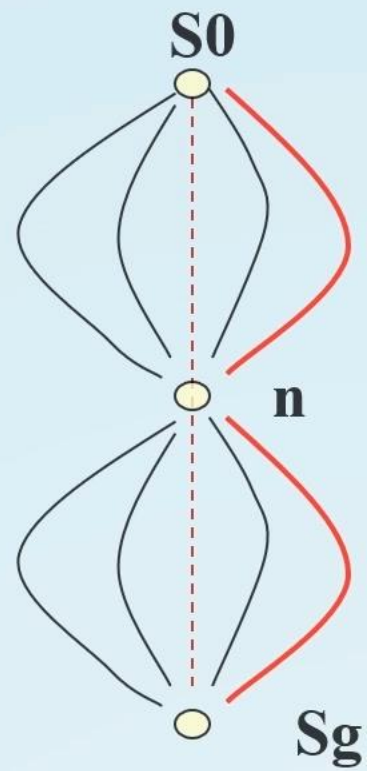
- **盲目式搜索的不足：**
 - 效率低，耗费过多的计算空间与时间

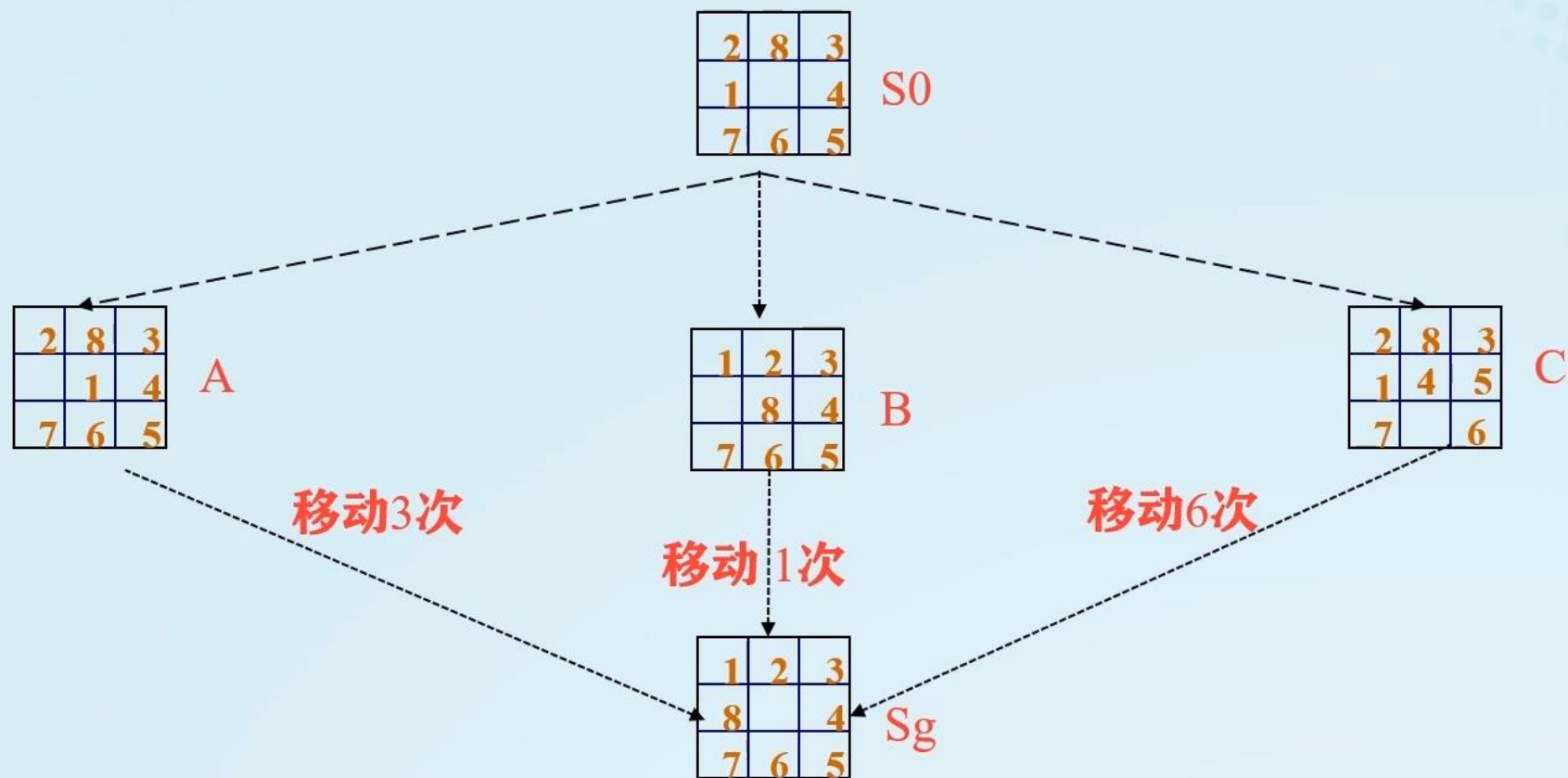


盲目式搜索



启发式搜索







启发式信息

- 启发性信息是指那种与具体问题求解过程有关的，并可指导搜索过程朝着**最有希望方向前进**的控制信息。
- 启发式就是要猜测：
 - 从节点 n 开始，找到最优解的可能性有多大？
 - 从起始节点开始，经过节点 n ，到达目标节点的最佳路径的费用是多少？



启发式搜索

- 启发式搜索是利用与问题有关的启发性信息，并以这些启发性信息指导的搜索的问题求解过程。
 - 需定义一个评价函数，对当前的搜索状态进行评估，找出一个最有希望的节点来扩展。
 - 重排OPEN表，选择最有希望的节点加以扩展
- 种类: A、A*算法



A 算法

- 估价函数 $f(n)$: 估算节点“希望”程度的量度

$$f(n) = g(n) + h(n)$$

从起始状态到当前
状态 n 的代价

从当前状态到目标状态的
估计代价 (启发函数)

A算法

局部择优 (尼尔逊, 1964)

有序搜索 (拉斐尔, 1967)



局部择优搜索(瞎子爬山法)

➤ **搜索过程**如下:

- (4) 考查节点 n 是否为目标节点, 若是, 则求得了问题的解, 退出;
- (5) 若节点 n 不可扩展, 则转第(2)步;
- (6) 扩展节点 n , 用估价函数 $f(x)$ **计算每个子节点的估价值, 并按估价值从小到大的顺序依次放入OPEN表的首部**, 并为每一个子节点都配置指向父节点的指针, 转第(2)步。



有序搜索

- **有序搜索**，也称最好优先搜索 / 全局择优，选择 OPEN 表上**具有最小 f 值的节点**作为下一个要扩展的节点。

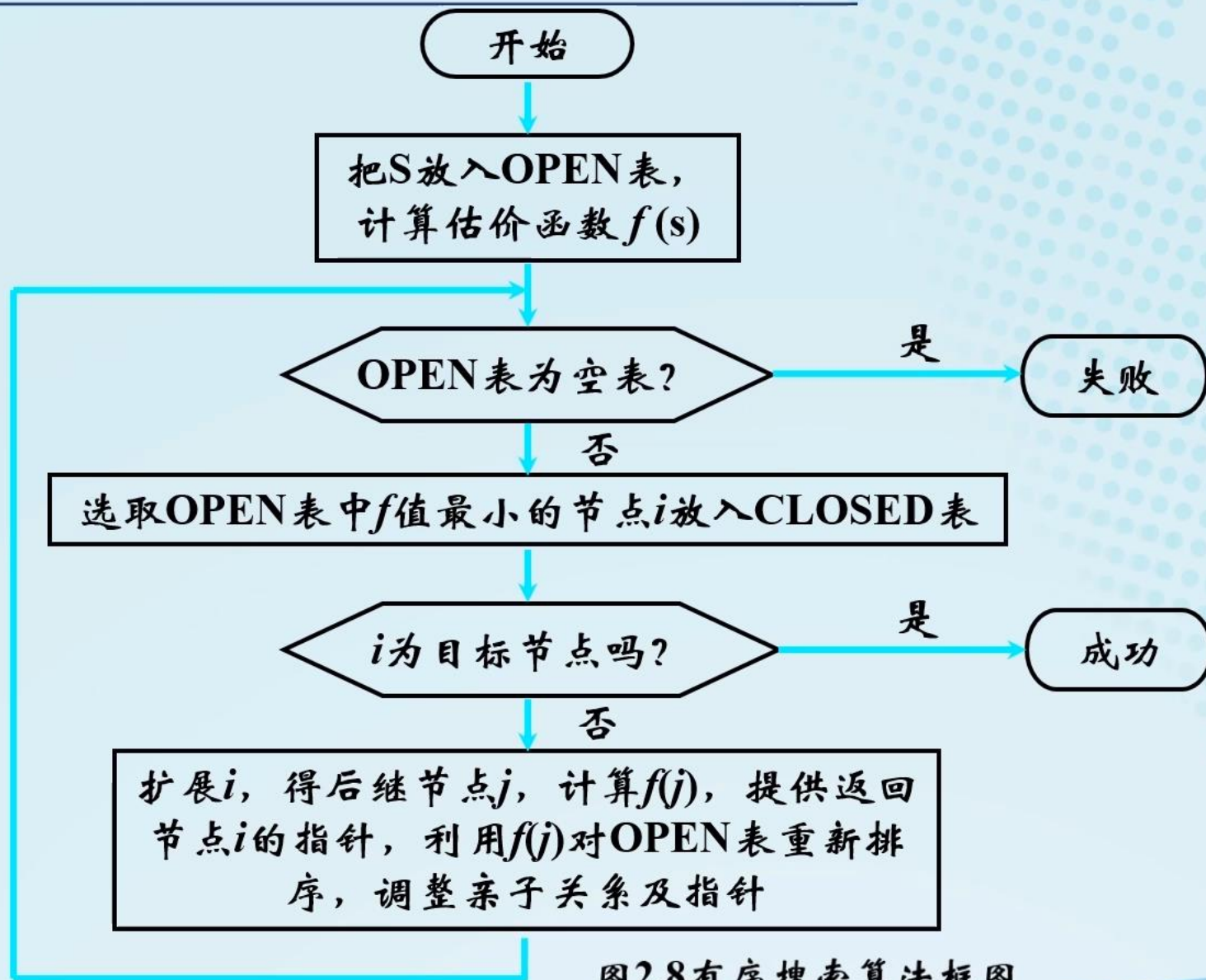


图2.8有序搜索算法框图



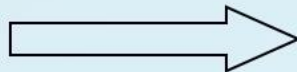
有序搜索的一个例子

定义**评价函数**:

- $f(n) = g(n) + h(n)$
- $g(n)$ 为从初始节点到当前节点的路径长度 (**层数**)
- $h(n)$ 为当前节点“**不在位**”的将牌数

2	8	3
1	6	4
7		5

S



1	2	3
8		4
7	6	5

g

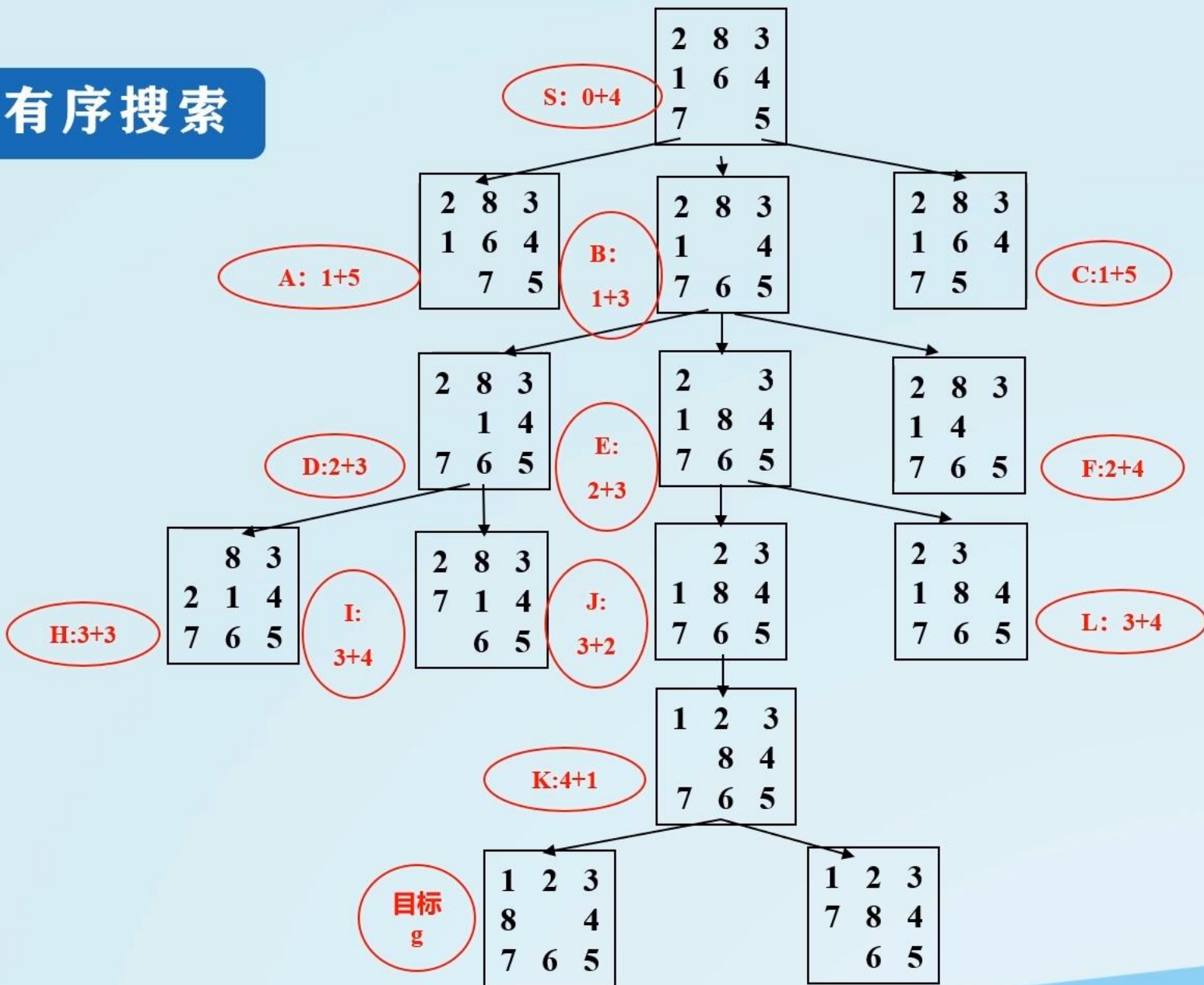
$$g(S)=0$$

$$h(S)=4$$

$$f(S)=0+4=4$$



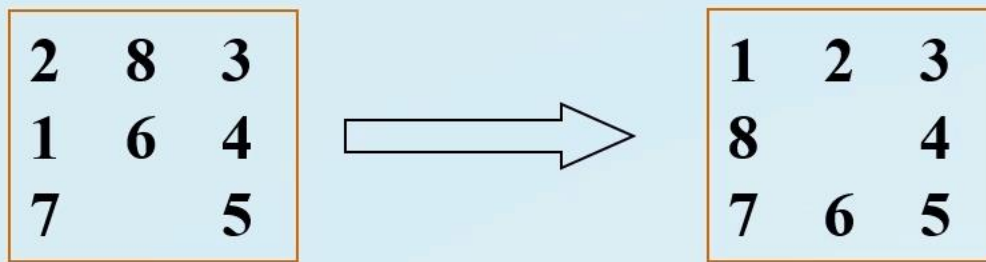
有序搜索



1	2	3
8		4
7	6	5



有序搜索



	宽度搜索	深度搜索（界为5）	A算法
扩展节点	26	18	6
生成节点	46	34	13



西安电子科技大学
XIDIAN UNIVERSITY



人工智能导论

启发式搜索（下）

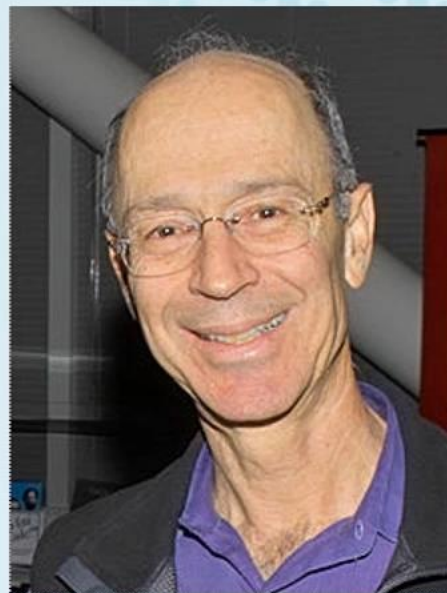
主讲人：刘若辰

西安电子科技大学 人工智能学院



A* 算法

- 1968年，彼得·哈特对A算法进行了很小的修改，并证明了当估价函数满足一定的限制条件时，算法一定可以找到最优解。
- 隶属于A算法，其特点在于对估价函数的定义上



彼得·哈特

➤ A*算法的限制条件

$$f(n) = \underline{g(n)} + \underline{h(n)}$$

大于0

要满足一定条件

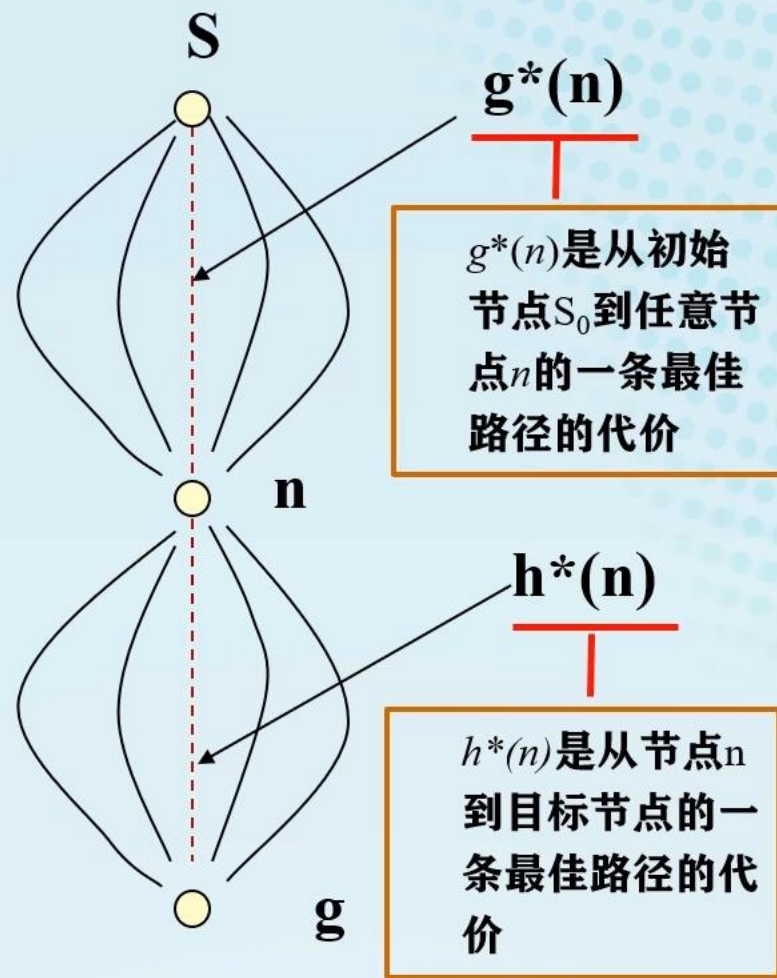


A* 算法

- 把OPEN表中的节点按估价函数的值从小到大进行排序

$$f(n) = g(n) + h(n)$$

- $g(n)$ 是对 $g^*(n)$ 的估计, $g(n) \geq g^*(n)$
- $h(n)$ 是 $h^*(n)$ 的下界, 即对所有的 n , $h(n) \leq h^*(n)$





A* 算法

➤ 定义2.1

在图搜索过程中，如果第8步的重排OPEN表是依据 $f(n)=g(n)+h(n)$ 进行的，则称该过程为A算法。

➤ 定义2.2

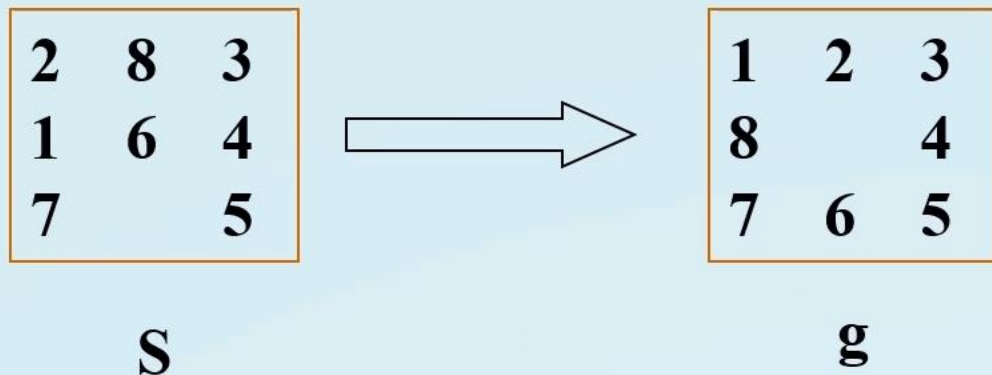
在A算法中，如果对所有的n存在 $h(n) \leq h^*(n)$ ，则称 $h(n)$ 为 $h^*(n)$ 的下界，它表示某种偏于保守的估计。

➤ 定义2.3

采用 $h^*(n)$ 的下界 $h(n)$ 为启发函数的A算法称为A*算法。 $h(n)=0$ 时，A*算法就变为等代价搜索算法。



A* 算法



➤ 方案1. $f(n) = g(n) + h_1(n)$

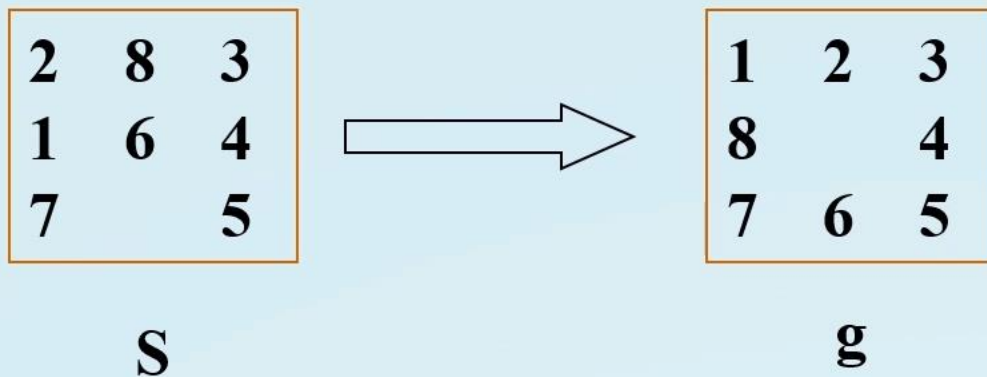
$g(n)$: 节点 n 的层数

$h_1(n)$: 表示“不在位”的将牌数

显然 $h_1(n) \leq h^*(n)$



A* 算法



➤ **方案2.** $f(n) = g(n) + h_2(n)$

$g(n)$: 节点n的层数

$h_2(n)$: 节点n的每一将牌与其目标位置之间的距离总和



A* 算法

➤ 八数码问题

—— $h_1(n)$ = “不在位”的将牌数

—— $h_2(n)$ = 将牌“不在位”的距离和

	1	2	3	
	2	8	3	
8	1	6	4	4
	7		5	5
	7	6		

不在位
将牌数

将牌 1 1
将牌 2 1
将牌 6 1
将牌 8 2

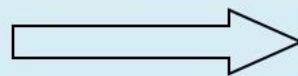
$$h_1(S) = 4$$

$$h_2(S) = 5$$

显然 $0 \leq h_1(n) \leq h_2(n) \leq h^*(n)$

2	8	3
1	6	4
7		5

S

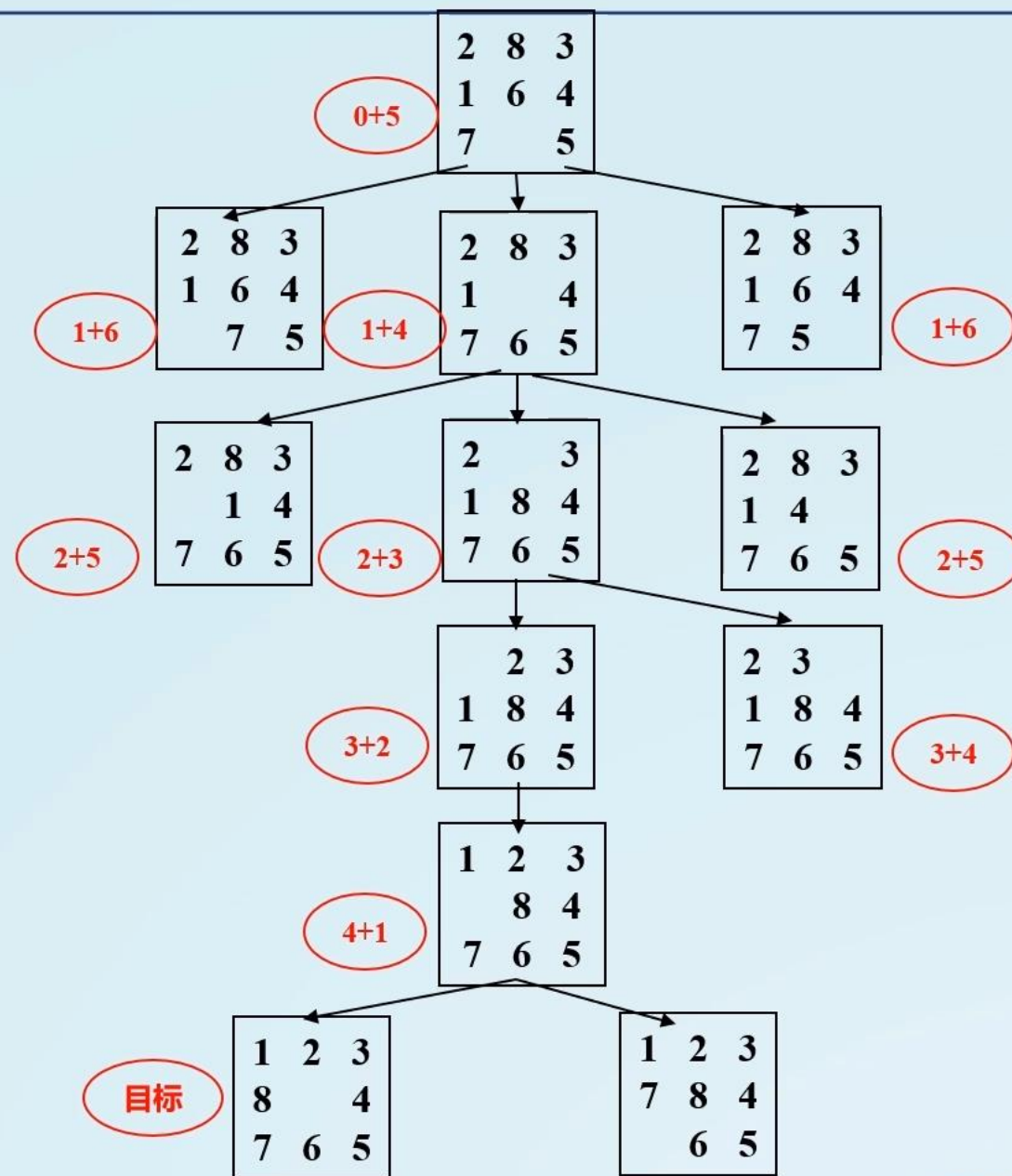


1	2	3
8		4
7	6	5

g



A* 算法



1	2	3
8		4
7	6	5



A* 算法

➤ 本题的启发函数的比较结果：

启发函数	$h(n)=0$	$h(n)=h_1(n)$	$h(n)=h_2(n)$
扩展节点	26	6	5
生成节点	46	13	11

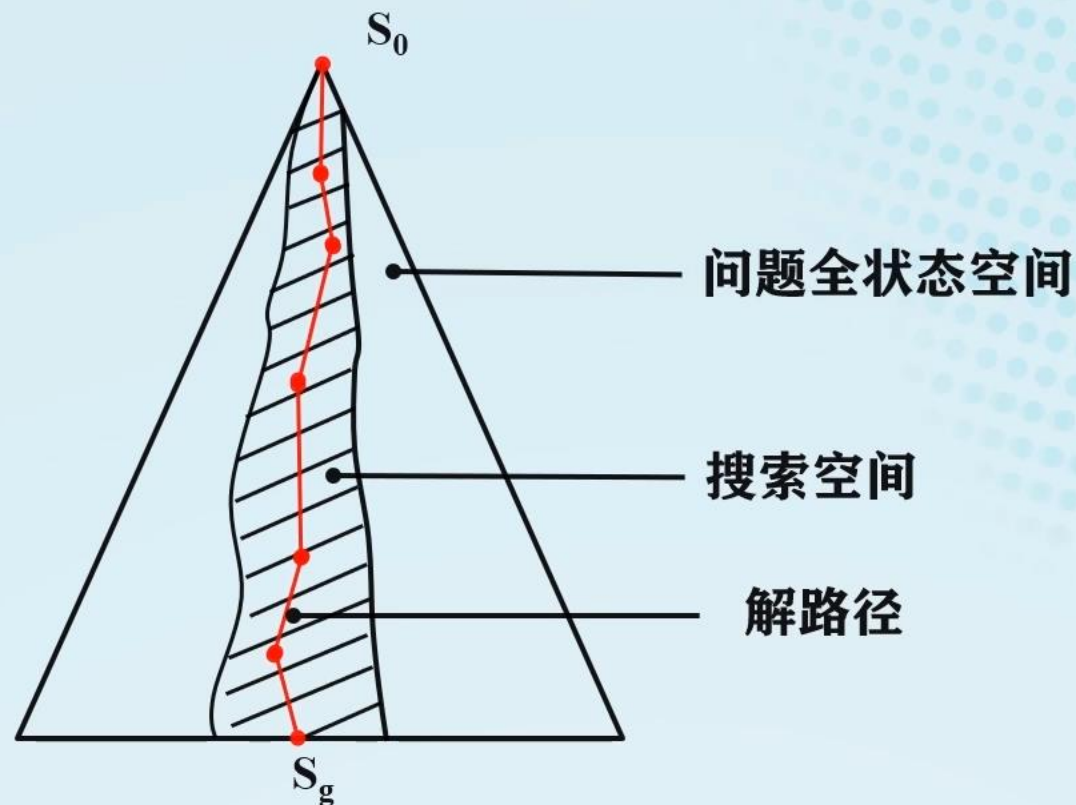
➤ A*算法的搜索效率在很大程度上取决于 $h(n)$ ，在满足 $h(n) \leq h^*(n)$ 的前提下， $h(n)$ 的值越大越好



衡量一个搜索策略性能的准则

➤ 搜索方法好的标准, 一般认为有3个:

- (1) 问题有解能否找到
- (2) 搜索空间小
- (3) 解最佳





状态空间的图搜索策略总结

